



HOPL IV

The Fourth ACM SIGPLAN History of Programming Languages Conference

20–22 June 2021

Co-Virtual with PLDI 2021

# A History of the Groovy Programming Language

Dr Paul King

*OCI Groovy Lead*

*V.P. and PMC Chair Apache Groovy*

*@paulk\_asert*



# The Groovy vision

# Vision: James Strachan, August 2003

*“So, I’ve been musing a little while if it’s time the Java platform had its own dynamic language designed from the ground up to work real nice with existing code; creating/extending objects that normal Java can use and vice versa.*

*Python/Jython’s a pretty good basis, add the nice stuff from Ruby and maybe sprinkle on some AOP features and we could have a really Groovy new language for scripting Java objects, writing test cases and who knows, even doing real development in it.*

*We’re starting simple with the nice tuples, sequences, maps from python & closures from ruby and being concise & dynamically typed with a java-look-and-feel though.*

*Where we end up is anyone’s guess right now.”*



# Vision: “*Just like Java only better*”

- **Java “*look and feel*”**
- **As much a companion to Java as a competitor**
  - Not about replacing Java but broadening the set of tasks where a Java developer could be productive
- **Improve Java perceived weaknesses/pain points**
  - Data scientists coding algorithms (or other non-dev coders)
    - *Preferred simplicity offered by Python or R*
  - Java developers
    - *Wanted less boilerplate code and better extensibility & productivity*
  - Smalltalk programmers forced to use the more popular Java
    - *Lamented the power and fun offered by Smalltalk*
  - Functional programmers
    - *Wanted better support for functional idioms and style*
  - Those scripting operating system activities (file munging, text processing, ...)
    - *Shell scripts or Perl considered more terse*

# Better Scripting: Java regex example

Leverage JVM for some regex code instead of using  
Shell script, Perl, AWK, Sed, Grep, ...

# Better Scripting: Java regex example

```
1 import org.junit.Test;
2 import java.util.regex.Matcher;
3 import java.util.regex.Pattern;
4 import static junit.framework.Assert.assertEquals;
5
6 public class RegexTest {
7     @Test
8     public void testRegexMatching() {
9         Pattern regex = Pattern.compile("(\\S+):(\\S+)");
10        Matcher matcher = regex.matcher("a:1 b:2 c:3");
11        int match = 0;
12        while (matcher.find()) {
13            match++;
14            if (match == 2) {
15                String full = matcher.group(0);
16                assertEquals("b:2", full);
17                String key = matcher.group(1);
18                assertEquals("b", key);
19                String value = matcher.group(2);
20                assertEquals("2", value);
21            }
22        }
23        assertEquals(3, match);
24    }
25 }
```

Listing 1. Java regular expression matching

# Better scripting: Groovy regex example

```
1 import org.junit.Test;
2 import java.util.regex.Matcher;
3 import java.util.regex.Pattern;
4 import static org.junit.Assert.*;
5
6 public class RegexTest {
7     @Test
8     public void testRegex() {
9         Pattern pattern = Pattern.compile("a:(\\S+) b:(\\S+) c:(\\S+)");
10        Matcher matcher = pattern.matcher("a:1 b:2 c:3");
11        int match = 0;
12        while (matcher.find()) {
13            match++;
14            if (match == 2) {
15                String full = matcher.group(0);
16                assertEquals("b:2", full);
17                String key = matcher.group(1);
18                assertEquals("b", key);
19                String value = matcher.group(2);
20                assertEquals("2", value);
21            }
22        }
23        assertEquals(3, match);
24    }
25 }
```

Listing 1. Java regular expression matching

```
1 def (first, second, third) = 'a:1 b:2 c:3' =~ /(\S+):(\S+)/
2 def (full, key, value) = second
3 assert full == 'b:2' && key == 'b' && value == '2'
```

Listing 2. Groovy regular expression matching

# Better scripting: Groovy regex example

```
1 import org.junit.Test;
2 import java.util.regex.Matcher;
3 import java.util.regex.Pattern;
4 import static org.junit.Assert.*;
5
6 public class RegexTest {
7     @Test
8     public void testRegex() {
9         Pattern pattern = Pattern.compile("a:1 b:2 c:3");
10        Matcher matcher = pattern.matcher("a:1 b:2 c:3");
11        int match = 0;
12        while (matcher.find()) {
13            match++;
14            if (match == 2) {
15                String full = matcher.group(0);
16                assertEquals("b:2", full);
17                String key = matcher.group(1);
18                assertEquals("b", key);
19                String value = matcher.group(2);
20                assertEquals("2", value);
21            }
22        }
23        assertEquals(3, match);
24    }
25 }
```

Listing 1. Java regular expression matching

```
1 def (first, second, third) = 'a:1 b:2 c:3' =~ /(\S+):(\S+)/
2 def (full, key, value) = second
3 assert full == 'b:2' && key == 'b' && value == '2'
```

Listing 2. Groovy regular expression matching

- Regex find operator syntactic sugar
- Regex friendly string variants
- Multi-assignment
- First class assertions
- Indexing syntactic sugar
- Operator overloading
- Library enhancements
- Extensions/metaprogramming



# Better for Data Scientists

Write some JVM code to perform some matrix manipulations

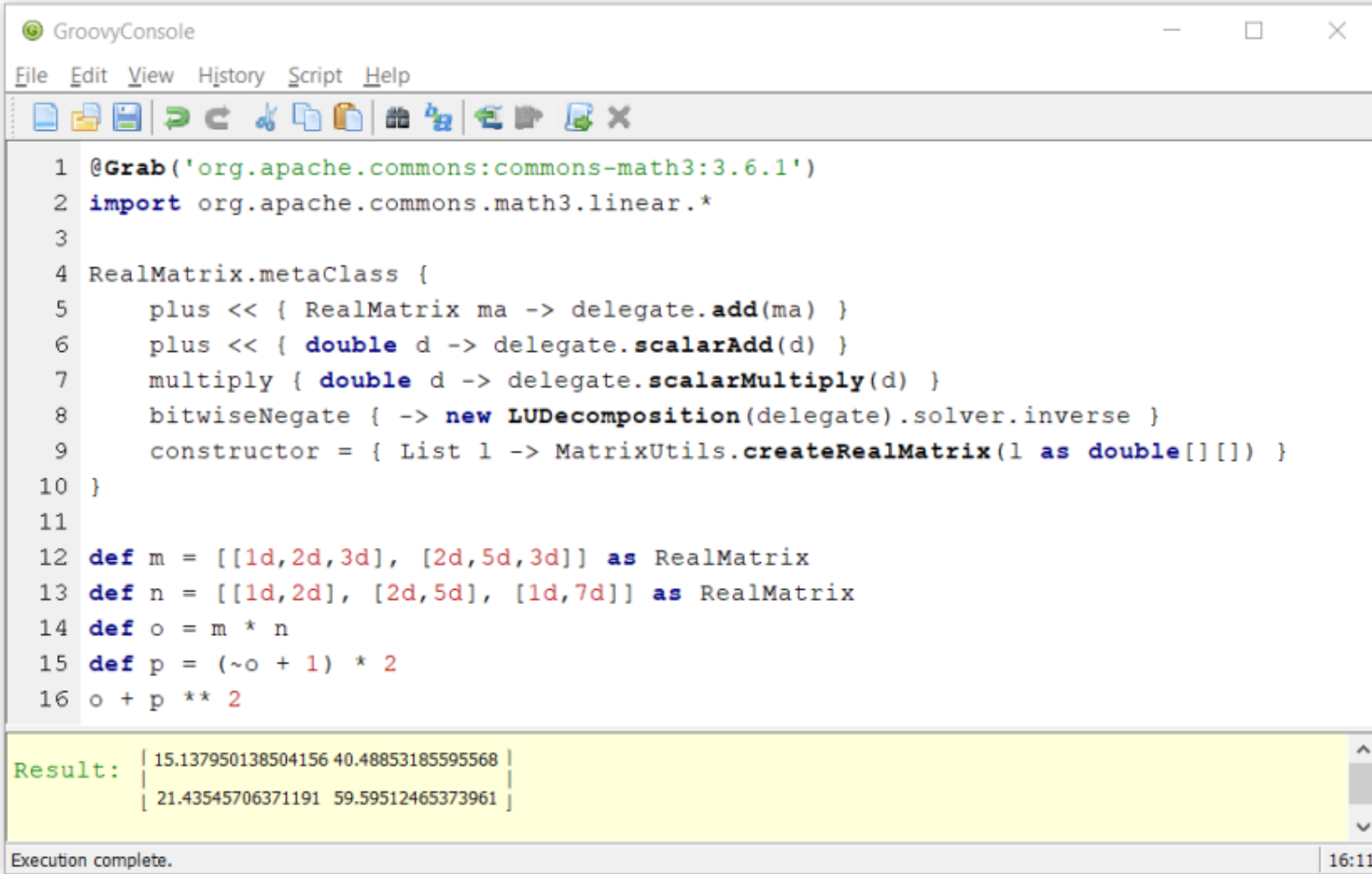
# Better for Data Scientists

```
1 import org.apache.commons.math3.linear.*;
2
3 public class MatrixMain {
4     public static void main(String[] args) {
5         double[][] matrixData = { {1d,2d,3d}, {2d,5d,3d}};
6         RealMatrix m = MatrixUtils.createRealMatrix(matrixData);
7         double[][] matrixData2 = { {1d,2d}, {2d,5d}, {1d, 7d}};
8         RealMatrix n = new Array2DRowRealMatrix(matrixData2);
9         RealMatrix o = m.multiply(n);
10        // Invert p, using LU decomposition
11        RealMatrix oInverse = new LUDecomposition(o).getSolver().getInverse();
12        RealMatrix p = oInverse.scalarAdd(1d).scalarMultiply(2d);
13        RealMatrix q = o.add(p.power(2));
14        System.out.println(q);
15    }
16 }
```

Listing 3. Java use of RealMatrix

```
Array2DRowRealMatrix{{15.1379501385,40.488531856},{21.4354570637,59.5951246537}}
```

# Better for Data Scientists



```
GroovyConsole
File Edit View History Script Help

1 @Grab('org.apache.commons:commons-math3:3.6.1')
2 import org.apache.commons.math3.linear.*
3
4 RealMatrix.metaClass {
5     plus << { RealMatrix ma -> delegate.add(ma) }
6     plus << { double d -> delegate.scalarAdd(d) }
7     multiply { double d -> delegate.scalarMultiply(d) }
8     bitwiseNegate { -> new LUdecomposition(delegate).solver.inverse }
9     constructor = { List l -> MatrixUtils.createRealMatrix(l as double[][]) }
10 }
11
12 def m = [[1d,2d,3d], [2d,5d,3d]] as RealMatrix
13 def n = [[1d,2d], [2d,5d], [1d,7d]] as RealMatrix
14 def o = m * n
15 def p = (~o + 1) * 2
16 o + p ** 2

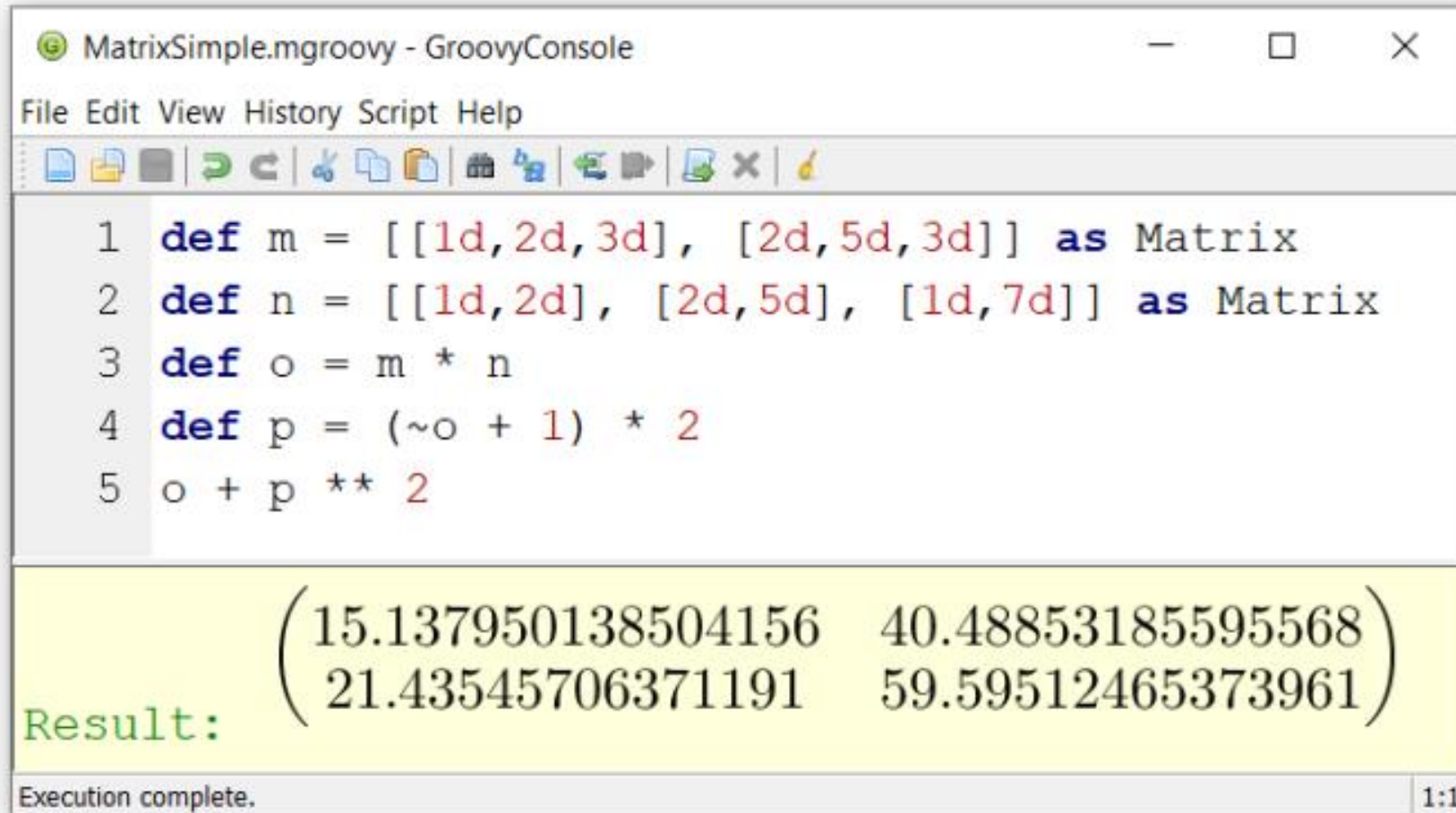
Result: [ 15.137950138504156 40.48853185595568 |
         [ 21.43545706371191 59.59512465373961 ]

Execution complete. 16:11
```

- Utilise Grab
- Metaprogramming
- Operator overloading
- Extensible tooling through output transform

Fig. 1. Matrix programming environment: Take 1

# Better for Data Scientists



The screenshot shows a window titled "MatrixSimple.mgroovy - GroovyConsole". The menu bar includes "File", "Edit", "View", "History", "Script", and "Help". The toolbar contains icons for file operations and execution. The code editor displays five lines of Groovy code:

```
1 def m = [[1d, 2d, 3d], [2d, 5d, 3d]] as Matrix
2 def n = [[1d, 2d], [2d, 5d], [1d, 7d]] as Matrix
3 def o = m * n
4 def p = (~o + 1) * 2
5 o + p ** 2
```

Below the code, the result is displayed in a yellow box:

Result: 
$$\begin{pmatrix} 15.137950138504156 & 40.48853185595568 \\ 21.43545706371191 & 59.59512465373961 \end{pmatrix}$$

The status bar at the bottom indicates "Execution complete." and "1:1".

- Hide Grab and metaprogramming and import aliasing through script integration
- Improved output transform with help from a  $\text{\TeX}$  library

Fig. 2. Matrix programming environment: Take 2

# Better for Data Scientists

```
MatrixSimple.mgroovy - GroovyConsole
File Edit View History Script Help
[Icons]
1 def m = [[1d,2d,3d], [2d,5d,3d]] as Matrix
2 def n = [[1d,2d], [2d,5d], [1d,7d]] as Matrix
3 def o = m * n
4 def p = (~o + 1) * 2
5 o + p ** 2

Result: (15.137950138504156 40.48853185595568)
        (21.43545706371191 59.59512465373961)
Execution complete. 1:1
```

Fig. 2. Matrix programming environment: Take 2

```
In [*]: p0 = priceModel.predict([0] as double[])
pMax = priceModel.predict([maxSqftLiving] as double[])
plot = new Plot(title: 'Price x Sqft living', xLabel: 'Sqft living', yLabel: 'Price')
plot << new Points(x: records.column('sqft_living').toList(),
                  y: records.column('price').toList())
plot << new Line(x: [0, maxSqftLiving], y: [p0, pMax])
```



This is much better but we can improve by considering multiple features.

```
In [*]: cols = ['sqft_living', 'bathrooms', 'grade', 'view', 'bedrooms',
               'sqft_above', 'yr_renovated', 'waterfront']
priceModel = new OLS(records.select(*cols + ['price']).smile().numericDataset('price'))
```

Jupyter/BeakerX integration

# And not just Data Scientists...

```
apply plugin: 'java'

repositories {
    mavenCentral()
}

dependencies {
    compile 'log4j:log4j:1.2.17'
    testCompile 'junit:junit:4.13.2'
}
```

Any “developer” on the periphery

- Someone maintaining a project build file

# And not just Data Scientists...

```
def "maximum of #a and #b is #c"() {  
  expect:  
    Math.max(a, b) == c  
  
  where:  
    a | b | | c  
    3 | 5 | | 5  
    7 | 0 | | 7  
    0 | 0 | | 0  
}
```

Any “developer” on  
the periphery

- Someone writing  
tests

# And not just Data Scientists...

```
if (JobGrade <= 3) {  
    return CommissionPercentage < 0.40  
} else {  
    return CommissionPercentage < 0.60  
}
```

Any “developer” on  
the periphery

- Someone writing  
business rules



# And not just Data Scientists...

```
regimen.take drug: CQ, qty: 1, at: 2.d

CRT = protein("CRT")
CRT.mutatingAmino 76, Lys, Thr

cqEffect = effect(
    name: "General",
    formula: {3.8 / (1 + km1/cq) },
    parameters: [km1: 68.0]
)
```

Any “developer” on the periphery

- Bioinformatics researchers exploring a drug regimen model  
(*Chloroquine resistance*)

# And not just Data Scientists...

```
from p in persons
groupby p.gender
orderby count(p.name)
select p.gender, count(p.name)
```

```
from n in numbers
where n > 0 && n <= 3
select n * 2
```

Any “developer” on  
the periphery

- Database guru  
searching in a  
collection  
*(Groovy 4's Language  
integrated query)*

# And not just Data Scientists...

```
show = { println it }  
square_root = { Math.sqrt(it) }  
  
def please(action) {  
  [the: { what ->  
    [of: { n -> action(what(n)) }}]  
}
```

**please show the square\_root of 100**

*// ==> 10.0*

*// equivalent to:*

*// please(show).the(square\_root).of(100)*

*// Japanese DSL using GEP3 rules*

```
Object.metaClass.を =  
  Object.metaClass.の = { clos ->  
    clos(delegate) }
```

```
まず = { it }  
表示する = { println it }  
平方根 = { Math.sqrt(it) }
```

**まず 100 の平方根 を 表示する**

*// First, show the square root of 100*

*// ==> 10.0*

Any “developer” on the periphery

- In general, any scenario where one of the intended readers is not a core developer

# And not just Data Scientists...

```
show = { println it }  
square_root = { Math.sqrt(it) }  
  
def please(action) {  
  [the: { what ->  
    [of: { n -> action(what(n)) }}]  
}]  
}
```

**please show the square\_root of 100**

*// ==> 10.0*

*// equivalent to:*

*// please(show).the(square\_root).of(100)*

*// Japanese DSL using GEP3 rules*

Object.metaClass.を =

Object.metaClass.の = { clos ->  
 clos(delegate) }

まず = { it }

表示する = { println it }

平方根 = { Math.sqrt(it) }

**まず 100 の平方根 を 表示する**

*// First, show the square root of 100*

*// ==> 10.0*

Any “developer” on  
the periphery

- In general, any scenario where one of the intended readers is not a core developer

Support the creation of mini “*Domain Specific Languages*” (DSLs)

# Better for Java Developers

Word count for File

# Better for Java Developers

## Word count for File as a utility class

```
import java.io.*;

public class FileUtil {
    public static int getWordCount(File file) throws IOException {
        FileInputStream fis = new FileInputStream(file);
        InputStreamReader isr = new InputStreamReader(fis);
        BufferedReader reader = new BufferedReader(isr);
        int wordCount = 0;
        String line;
        while ((line = reader.readLine()) != null) {
            String[] words = line.split("\\w+");
            wordCount += words.length;
        }
        return wordCount;
    }
}
```

```
FileUtil.getWordCount(new File('myFile.txt'))
```

# Better for Java Developers

## Word count for File as a utility class (recent Java versions)

```
import java.io.*;

public class FileUtil {
    public static int getWordCount(File file) throws IOException {
        FileInputStream fis = new FileInputStream(file);
        InputStreamReader isr = new InputStreamReader(fis);
        BufferedReader reader = new BufferedReader(isr);
        int wordCount = 0;
        String line;
        while ((line = reader.readLine()) != null) {
            String[] words = line.split("\\s+");
            wordCount += words.length;
        }
        return wordCount;
    }
}

import java.io.File;
import java.io.IOException;
import java.nio.charset.Charset;
import java.nio.file.Files;
import java.util.Arrays;

public class FileUtil {
    public static long getWordCount(File file) throws IOException {
        return Files.lines(file.toPath(), Charset.forName("ISO-8859-1"))
            .flatMap(line -> Arrays.stream(line.trim().split("\\w+")))
            .count();
    }
}

FileUtil.getWordCount(new File('myFile.txt'))
```

# Better for Java Developers

## Word count for File as Groovy extension

```
import java.io.*;
```

```
File.metaClass.getWordCount = { delegate.text.split(/\w+/).size() }
```

```
public class FileUtil {
```

```
    public static int getWordCount(File file) throws IOException {
```

```
        FileInputStream fis = new FileInputStream(file);
```

```
        InputStreamReader isr = new InputStreamReader(fis);
```

```
        BufferedReader reader = new BufferedReader(isr);
```

```
        int wordCount = 0;
```

```
        String line;
```

```
        while ((line = reader.readLine()) != null) {
```

```
            String[] words = line.split("\\w+");
```

```
            wordCount += words.length;
```

```
        }
```

```
        return wordCount;
```

```
    }
```

```
}
```

```
new File('myFile.txt').wordCount
```

```
FileUtil.getWordCount(new File('myFile.txt'))
```



# Better for Java Developers

Runtime metaprogramming example: Grails Book domain object

# Better for Java Developers

Runtime metaprogramming example: Grails Book domain object

```
class Book {  
    String title  
    String author  
    Boolean paperback  
}
```

```
Book.findByTitleAndAuthor("The Sum of All Fears", "Tom Clancy")  
Book.findByTitleLike("%Hobbit%")  
Book.findNotPaperbackByAuthor("Douglas Adams")
```

# Better for Java Developers

Compile-time metaprogramming example:

- Book domain object
  - Immutable
  - Sortable in various ways
  - Externalizable to persist or transfer via byte streams
- Approach
  - Follow best practice idioms
  - Follow JavaBean conventions
  - Use appropriate design patterns
  - View to maintainability

# Better for Java Developers

First attempt

- Essentially data only for now
- Still mutable

```
class Book {  
    List<String> authors  
    String title  
    Date publicationDate  
}
```

# Better for Java Developers

## Equivalent Java

```
class Book {  
  
    List<String> authors  
  
    String title  
  
    Date publicationDate  
  
}
```

```
import java.util.Date;  
import java.util.List;  
  
public class Book {  
  
    private String title;  
    private List<String> authors;  
    private Date publicationDate;  
  
    public String getTitle() {  
        return title;  
    }  
  
    public void setTitle(String title) {  
        this.title = title;  
    }  
  
    public List<String> getAuthors() {  
        return authors;  
    }  
  
    public void setAuthors(List<String> authors) {  
        this.authors = authors;  
    }  
  
    public Date getPublicationDate() {  
        return publicationDate;  
    }  
  
    public void setPublicationDate(Date publicationDate) {  
        this.publicationDate = publicationDate;  
    }  
}
```

# Better for Java Developers

## Add JavaBean indexed getter

```
class Book {  
    @IndexedProperty  
    List<String> authors  
  
    String title  
  
    Date publicationDate  
}
```

```
import java.util.*;  
  
public class Book {  
    private List<String> authors;  
    private String title;  
    private Date publicationDate;  
  
    public void setAuthors(int index, String value) {  
        authors.set(index, value);  
    }  
  
    public String getAuthors(int index) {  
        return authors.get(index);  
    }  
  
    public List<String> getAuthors() {  
        return authors;  
    }  
  
    public void setAuthors(List<String> authors) {  
        this.authors = authors;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public void setTitle(String title) {  
        this.title = title;  
    }  
  
    public Date getPublicationDate() {  
        return publicationDate;  
    }  
  
    public void setPublicationDate(Date publicationDate) {  
        this.publicationDate = publicationDate;  
    }  
}
```

# Better for Java Developers

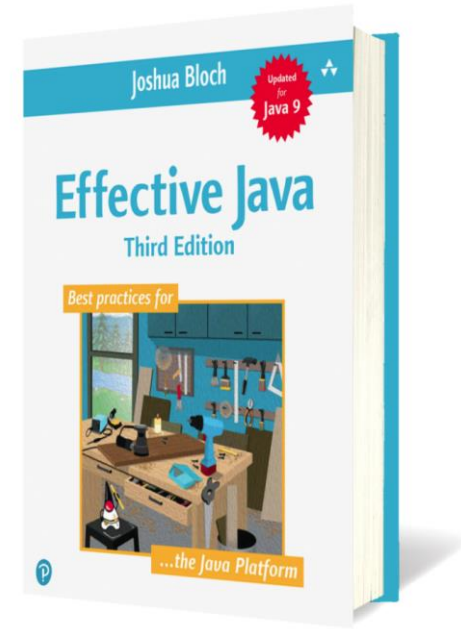
Add immutability  
conventions

```
class Book {  
    @IndexedProperty  
    List<String> authors  
  
    String title  
  
    Date publicationDate  
}
```

# Better for Java Developers

Add immutability conventions

```
class Book {  
    @IndexedProperty  
    List<String> authors  
  
    String title  
  
    Date publicationDate  
}
```



- Don't provide mutators
- Ensure that no methods can be overridden
- Make all fields final & private
- Ensure exclusive access to any mutable components
- Optionally provide equals and *hashCode* methods
- Optionally provide *toString* method

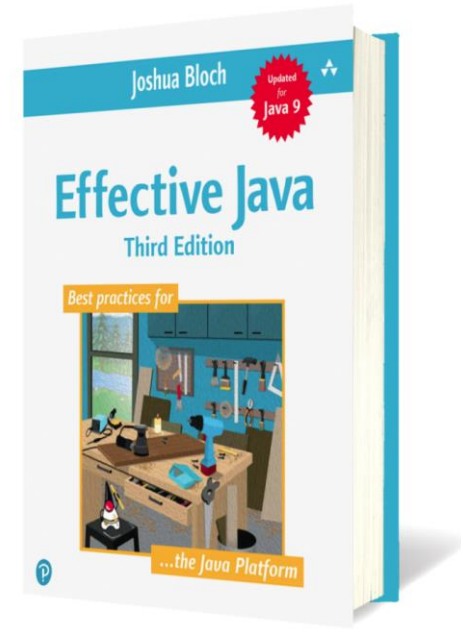


# Better for Java Developers

Add immutability conventions

```
@Immutable(copyWith = true)
```

```
class Book {  
    @IndexedProperty  
    List<String> authors  
  
    String title  
  
    Date publicationDate  
}
```



- Don't provide mutators
- Ensure that no methods can be overridden
- Make all fields final & private
- Ensure exclusive access to any mutable components
- Optionally provide equals and *hashCode* methods
- Optionally provide *toString* method

# Better for Java Developers

## Add immutability conventions

@Immutable(copyWith = true)

```
class Book {  
    @IndexedProperty  
    List<String> authors  
  
    String title  
  
    Date publicationDate  
}
```

```
// imports not shown  
  
public class Book {  
    private String $toString;  
    private int $hashCode;  
    private final List<String> authors;  
    private final String title;  
    private final Date publicationDate;  
  
    public Book(List<String> authors, String title, Date publicationDate) {  
        if (authors == null) {  
            this.authors = null;  
        } else {  
            if (authors instanceof Cloneable) {  
                List<String> authorsCopy = (List<String>) ((ArrayList<?>) authors).clone();  
                this.authors = (List<String>) (authorsCopy instanceof SortedSet ? DefaultGroovyMethods.as  
                    : authorsCopy instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                    : authorsCopy instanceof Set ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                    : authorsCopy instanceof Map ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                    : authorsCopy instanceof List ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                    : DefaultGroovyMethods.asImmutable(authorsCopy));  
            } else {  
                this.authors = (List<String>) (authors instanceof SortedSet ? DefaultGroovyMethods.asImm  
                    : authors instanceof Set ? DefaultGroovyMethods.asImmutable(authors)  
                    : authors instanceof Map ? DefaultGroovyMethods.asImmutable(authors)  
                    : authors instanceof List ? DefaultGroovyMethods.asImmutable(authors)  
                    : DefaultGroovyMethods.asImmutable(authors));  
            }  
        }  
        this.title = title;  
        if (publicationDate == null) { this.publicationDate = null; } else { this.publicationDate = (Date) pu  
    }  
  
    public Book(Map args) {  
        if (args == null) {  
            args = new HashMap();  
        }  
        ImmutableASTTransformation.checkPropNames(this, args);  
        if (args.containsKey("authors")) {  
            if (args.get("authors") == null) {  
                this.authors = null;  
            } else {  
                if (args.get("authors") instanceof Cloneable) {  
                    List<String> authorsCopy = (List<String>) ((ArrayList<?>) args.get("authors")).clone();  
                    this.authors = (List<String>) (authorsCopy instanceof SortedSet ? DefaultGroovyMethods.  
                        : authorsCopy instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                        : authorsCopy instanceof Set ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                        : authorsCopy instanceof Map ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                        : authorsCopy instanceof List ? DefaultGroovyMethods.asImmutable(authorsCopy)  
                        : DefaultGroovyMethods.asImmutable(authorsCopy));  
                } else {  
                    List<String> authors = (List<String>) args.get("authors");  
                    this.authors = (List<String>) (authors instanceof SortedSet ? DefaultGroovyMethods.asImm  
                        : authors instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authors)  
                        : authors instanceof Set ? DefaultGroovyMethods.asImmutable(authors)  
                        : authors instanceof Map ? DefaultGroovyMethods.asImmutable(authors)  
                        : authors instanceof List ? DefaultGroovyMethods.asImmutable(authors)  
                        : DefaultGroovyMethods.asImmutable(authors));  
                }  
            }  
        } else {  
            this.authors = null;  
        }  
        if (args.containsKey("title")) {  
            this.title = (String) args.get("title");  
        } else {  
            this.title = null;  
        }  
        if (args.containsKey("publicationDate")) {  
            if (args.get("publicationDate") == null) {  
                this.publicationDate = null;  
            } else {  
                this.publicationDate = (Date) ((Date) args.get("publicationDate")).clone();  
            }  
        } else {  
            this.publicationDate = null;  
        }  
    }  
}
```

```
public Book() {  
    this(new HashMap());  
}  
  
public String toString() {  
    StringBuilder _result = new StringBuilder();  
    boolean $toStringFirst = true;  
    if ($toStringFirst) {  
        $toStringFirst = false;  
    } else {  
        _result.append(", ");  
    }  
    _result.append(InvokerHelper.toString(this.getAuthors()));  
    if ($toStringFirst) {  
        $toStringFirst = false;  
    } else {  
        _result.append(", ");  
    }  
    _result.append(InvokerHelper.toString(this.getTitle()));  
    if ($toStringFirst) {  
        $toStringFirst = false;  
    } else {  
        _result.append(", ");  
    }  
    _result.append(InvokerHelper.toString(this.getPublicationDate()));  
    _result.append("");  
    if ($toString == null) {  
        $toString = _result.toString();  
    }  
    return $toString;  
}  
  
public int hashCode() {  
    if ($hashCode == 0) {  
        int _result = HashCodeHelper.initHash();  
        if (!this.getAuthors().equals(this)) {  
            _result = HashCodeHelper.updateHash(_result, this.getAuthors());  
        }  
        if (!this.getTitle().equals(this)) {  
            _result = HashCodeHelper.updateHash(_result, this.getTitle());  
        }  
        if (!this.getPublicationDate().equals(this)) {  
            _result = HashCodeHelper.updateHash(_result, this.getPublicationDate());  
        }  
        $hashCode = (int) _result;  
    }  
    return $hashCode;  
}  
  
public boolean canEqual(Object other) {  
    return other instanceof Book;  
}  
  
public boolean equals(Object other) {  
    if (other == null) {  
        return false;  
    }  
    if (this == other) {  
        return true;  
    }  
    if (!(other instanceof Book)) {  
        return false;  
    }  
    Book otherTyped = (Book) other;  
    if (!otherTyped.canEqual(this)) {  
        return false;  
    }  
    if (!this.getAuthors() == otherTyped.getAuthors()) {  
        return false;  
    }  
    if (!this.getTitle().equals(otherTyped.getTitle())) {  
        return false;  
    }  
    if (!this.getPublicationDate().equals(otherTyped.getPublicationDate())) {  
        return false;  
    }  
    return true;  
}  
  
public final Book copyWith(Map map) {  
    if (map == null || map.size() == 0) {  
        return this;  
    }  
    Boolean dirty = false;  
    HashMap construct = new HashMap();  
    if (map.containsKey("authors")) {  
        Object newValue = map.get("authors");  
        Object oldValue = this.getAuthors();  
        if (newValue != oldValue) {  
            oldValue = newValue;  
            dirty = true;  
        }  
        construct.put("authors", oldValue);  
    } else {  
        construct.put("authors", this.getAuthors());  
    }  
    if (map.containsKey("title")) {  
        Object newValue = map.get("title");  
        Object oldValue = this.getTitle();  
        if (newValue != oldValue) {  
            oldValue = newValue;  
            dirty = true;  
        }  
        construct.put("title", oldValue);  
    } else {  
        construct.put("title", this.getTitle());  
    }  
    if (map.containsKey("publicationDate")) {  
        Object newValue = map.get("publicationDate");  
        Object oldValue = this.getPublicationDate();  
        if (newValue != oldValue) {  
            oldValue = newValue;  
            dirty = true;  
        }  
        construct.put("publicationDate", oldValue);  
    } else {  
        construct.put("publicationDate", this.getPublicationDate());  
    }  
    return dirty == true ? new Book(construct) : this;  
}  
  
public String getAuthors(int index) {  
    return authors.get(index);  
}  
  
public List<String> getAuthors() {  
    return authors;  
}  
  
public final String getTitle() {  
    return title;  
}  
  
public final Date getPublicationDate() {  
    if (publicationDate == null) {  
        return publicationDate;  
    } else {  
        return (Date) publicationDate.clone();  
    }  
}
```

# Better for Java Developers

Add sorting and  
externalization

```
@Immutable(copyWith = true)
@Sortable(excludes = 'authors')
@AutoExternalize
class Book {
    @IndexedProperty
    List<String> authors

    String title

    Date publicationDate
}
```

# Better for Java Developers

```
// imports not shown
public class Book {

    private String $toString;
    private int $hashCode;
    private final List<String> authors;
    private final String title;
    private final Date publicationDate;
    private static final java.util.Comparator this$titleComparator;
    private static final java.util.Comparator this$publicationDateComparator;

    public Book(List<String> authors, String title, Date publicationDate) {
        if (authors == null) {
            this.authors = null;
        } else {
            if (authors instanceof Cloneable) {
                List<String> authorsCopy = (List<String>) ((ArrayList<?>) authors).clone();
                this.authors = (List<String>) (authorsCopy instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authorsCopy)
                    : authorsCopy instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authorsCopy)
                    : authorsCopy instanceof Set ? DefaultGroovyMethods.asImmutable(authorsCopy)
                    : authorsCopy instanceof Map ? DefaultGroovyMethods.asImmutable(authorsCopy)
                    : authorsCopy instanceof List ? DefaultGroovyMethods.asImmutable(authorsCopy)
                    : DefaultGroovyMethods.asImmutable(authorsCopy));
            } else {
                this.authors = (List<String>) (authors instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authors)
                    : authors instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authors)
                    : authors instanceof Set ? DefaultGroovyMethods.asImmutable(authors)
                    : authors instanceof Map ? DefaultGroovyMethods.asImmutable(authors)
                    : authors instanceof List ? DefaultGroovyMethods.asImmutable(authors)
                    : DefaultGroovyMethods.asImmutable(authors));
            }
        }
        this.title = title;
        if (publicationDate == null) {
            this.publicationDate = null;
        } else {
            this.publicationDate = (Date) publicationDate.clone();
        }
    }

    public Book(Map args) {
        if ( args == null) {
            args = new HashMap();
        }
        ImmutableASTTransformation.checkPropNames(this, args);
        if (args.containsKey("authors")) {
            if ( args.get("authors") == null) {
                this.authors = null;
            } else {
                if (args.get("authors") instanceof Cloneable) {
                    List<String> authorsCopy = (List<String>) ((ArrayList<?>) args.get("authors")).clone();
                    this.authors = (List<String>) (authorsCopy instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authorsCopy)
                        : authorsCopy instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authorsCopy)
                        : authorsCopy instanceof Set ? DefaultGroovyMethods.asImmutable(authorsCopy)
                        : authorsCopy instanceof Map ? DefaultGroovyMethods.asImmutable(authorsCopy)
                        : authorsCopy instanceof List ? DefaultGroovyMethods.asImmutable(authorsCopy)
                        : DefaultGroovyMethods.asImmutable(authorsCopy));
                } else {
                    List<String> authors = (List<String>) args.get("authors");
                    this.authors = (List<String>) (authors instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authors)
                        : authors instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authors)
                        : authors instanceof Set ? DefaultGroovyMethods.asImmutable(authors)
                        : authors instanceof Map ? DefaultGroovyMethods.asImmutable(authors)
                        : authors instanceof List ? DefaultGroovyMethods.asImmutable(authors)
                        : DefaultGroovyMethods.asImmutable(authors));
                }
            }
        }
        if (args.containsKey("title")) {this.title = (String) args.get("title");} else { this.title = null;}
        if (args.containsKey("publicationDate")) {
            if (args.get("publicationDate") == null) {
                this.publicationDate = null;
            } else {
                this.publicationDate = (Date) ((Date) args.get("publicationDate")).clone();
            }
        }
    }
}
```

```
public Book() {
    this (new HashMap());
}

public int compareTo(Book other){
    if (this == other){
        return 0;
    }
    Integer value = 0
    value = this.title <=> other.title
    if ( value != 0) {
        return value
    }
    value = this.publicationDate <=> other.publicationDate
    if ( value != 0) {
        return value
    }
    return 0
}

public static Comparator comparatorByTitle(){
    return this$titleComparator;
}

public static Comparator comparatorByPublicationDate(){
    return this$publicationDateComparator;
}

public String toString(){
    StringBuilder _result = new StringBuilder();
    boolean $toStringFirst = true;
    _result.append("Book(");
    $toStringFirst = false;
    } else {
        _result.append(", ");
    }
    _result.append(InvokerHelper.toString(this.getAuthors()));
    if ($toStringFirst) {
        $toStringFirst = false;
    } else {
        _result.append(", ");
    }
    _result.append(InvokerHelper.toString(this.getTitle()));
    if ($toStringFirst) {
        $toStringFirst = false;
    } else {
        _result.append(", ");
    }
    _result.append(InvokerHelper.toString(this.getPublicationDate()));
    _result.append(")");
    if ($toString == null) {
        $toString = _result.toString();
    }
    return $toString;
}

public int hashCode() {
    if ( $hashCode == 0) {
        int _result = hashCodeHelper.initHash();
        if (! (this.getAuthors().equals(this)) ) {
            _result = hashCodeHelper.updateHash(_result, this.getAuthors());
        }
        if (! (this.getTitle().equals(this)) ) {
            _result = hashCodeHelper.updateHash(_result, this.getTitle());
        }
        if (! (this.getPublicationDate().equals(this)) ) {
            _result = hashCodeHelper.updateHash(_result, this.getPublicationDate());
        }
        $hashCode = (int) _result;
    }
    return $hashCode;
}

public boolean canEqual(Object other) {
    return other instanceof Book;
}
}
```

```
public boolean equals(Object other) {
    if ( other == null) {
        return false;
    }
    if (this == other) {
        return true;
    }
    if (! (other instanceof Book)) {
        return false;
    }
    Book otherTyped = (Book) other;
    if (! (otherTyped.canEqual(this) ) ) {
        return false;
    }
    if (! (this.getAuthors() == otherTyped.getAuthors()) ) {
        return false;
    }
    if (! (this.getTitle().equals(otherTyped.getTitle()) ) ) {
        return false;
    }
    if (! (this.getPublicationDate().equals(otherTyped.getPublicationDate()) ) ) {
        return false;
    }
    return true;
}

public final Book copyWith(Map map) {
    if (map == null || map.size() == 0) {
        return this;
    }
    Boolean dirty = false;
    HashMap construct = new HashMap();
    if (map.containsKey("authors")) {
        Object newValue = map.get("authors");
        Object oldValue = this.getAuthors();
        if (newValue != oldValue) {
            oldValue = newValue;
            dirty = true;
        }
        construct.put("authors", oldValue);
    } else {
        construct.put("authors", this.getAuthors());
    }
    if (map.containsKey("title")) {
        Object newValue = map.get("title");
        Object oldValue = this.getTitle();
        if (newValue != oldValue) {
            oldValue = newValue;
            dirty = true;
        }
        construct.put("title", oldValue);
    } else {
        construct.put("title", this.getTitle());
    }
    if (map.containsKey("publicationDate")) {
        Object newValue = map.get("publicationDate");
        Object oldValue = this.getPublicationDate();
        if (newValue != oldValue) {
            oldValue = newValue;
            dirty = true;
        }
        construct.put("publicationDate", oldValue);
    } else {
        construct.put("publicationDate", this.getPublicationDate());
    }
    return dirty == true ? new Book(construct) : this;
}

public void writeExternal(ObjectOutput out) throws IOException {
    out.writeObject(authors);
    out.writeObject(title);
    out.writeObject(publicationDate);
}

public void readExternal(ObjectInput oin) throws IOException, ClassNotFoundException {
    authors = (List) oin.readObject();
    title = (String) oin.readObject();
    publicationDate = (Date) oin.readObject();
}
}
```

```
static {
    this$titleComparator = new Book$titleComparator();
    this$publicationDateComparator = new Book$publicationDateComparator();
}

public String getAuthors(int index) {
    return authors.get(index);
}

public List<String> getAuthors() {
    return authors;
}

public final String getTitle() {
    return title;
}

public final Date getPublicationDate() {
    if (publicationDate == null) {
        return publicationDate;
    } else {
        return (Date) publicationDate.clone();
    }
}

public int compare(java.lang.Object param0, java.lang.Object param1) {
    return -1;
}

private static class Book$titleComparator extends AbstractComparator<Book> {
    public Book$titleComparator() {}

    public int compare(Book arg0, Book arg1) {
        if (arg0 == arg1) {
            return 0;
        }
        if (arg0 != null && arg1 == null) {
            return -1;
        }
        if (arg0 == null && arg1 != null) {
            return 1;
        }
        return arg0.title <=> arg1.title;
    }

    public int compare(java.lang.Object param0, java.lang.Object param1) {
        return -1;
    }
}

private static class Book$publicationDateComparator extends AbstractComparator<Book> {
    public Book$publicationDateComparator() {}

    public int compare(Book arg0, Book arg1) {
        if (arg0 == arg1) {
            return 0;
        }
        if (arg0 != null && arg1 == null) {
            return -1;
        }
        if (arg0 == null && arg1 != null) {
            return 1;
        }
        return arg0.publicationDate <=> arg1.publicationDate;
    }

    public int compare(java.lang.Object param0, java.lang.Object param1) {
        return -1;
    }
}
}
```

# Better for Java Developers

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// imports not shown public class Book {      private String \$toString;     private int \$hashCode;     private final List&lt;String&gt; authors;     private final String title;     private final Date publicationDate;     private static final java.util.Comparator this\$titleComparator;     private static final java.util.Comparator this\$publicationDateComparator;      public Book(List&lt;String&gt; authors, String title, Date publicationDate) {         if (authors == null) {             this.authors = null;         } else {             if (authors instanceof Cloneable) {                 List&lt;String&gt; authorsCopy = (List&lt;String&gt;) ((ArrayList&lt;?&gt;) authors).clone();                 this.authors = (List&lt;String&gt;) (authorsCopy instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authorsCopy)                     : authorsCopy instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authorsCopy)                     : authorsCopy instanceof Set ? DefaultGroovyMethods.asImmutable(authorsCopy)                     : authorsCopy instanceof Map ? DefaultGroovyMethods.asImmutable(authorsCopy)                     : authorsCopy instanceof List ? DefaultGroovyMethods.asImmutable(authorsCopy)                     : DefaultGroovyMethods.asImmutable(authorsCopy));             } else {                 this.authors = (List&lt;String&gt;) (authors instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authors)                     : authors instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authors)                     : authors instanceof Set ? DefaultGroovyMethods.asImmutable(authors)                     : authors instanceof Map ? DefaultGroovyMethods.asImmutable(authors)                     : authors instanceof List ? DefaultGroovyMethods.asImmutable(authors)                     : DefaultGroovyMethods.asImmutable(authors));             }         }         this.title = title;         if (publicationDate == null) {             this.publicationDate = null;         } else {             this.publicationDate = (Date) publicationDate.clone();         }     }      public Book(Map args) {         if (args == null) {             args = new HashMap();         }         ImmutableASTTransformation.checkPropNames(this, args);         if (args.containsKey("authors")) {             if (args.get("authors") == null) {                 this.authors = null;             } else {                 if (args.get("authors") instanceof Cloneable) {                     List&lt;String&gt; authorsCopy = (List&lt;String&gt;) ((ArrayList&lt;?&gt;) args.get("authors")).clone();                     this.authors = (List&lt;String&gt;) (authorsCopy instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authorsCopy)                         : authorsCopy instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authorsCopy)                         : authorsCopy instanceof Set ? DefaultGroovyMethods.asImmutable(authorsCopy)                         : authorsCopy instanceof Map ? DefaultGroovyMethods.asImmutable(authorsCopy)                         : authorsCopy instanceof List ? DefaultGroovyMethods.asImmutable(authorsCopy)                         : DefaultGroovyMethods.asImmutable(authorsCopy));                 } else {                     List&lt;String&gt; authors = (List&lt;String&gt;) args.get("authors");                     this.authors = (List&lt;String&gt;) (authors instanceof SortedSet ? DefaultGroovyMethods.asImmutable(authors)                         : authors instanceof SortedMap ? DefaultGroovyMethods.asImmutable(authors)                         : authors instanceof Set ? DefaultGroovyMethods.asImmutable(authors)                         : authors instanceof Map ? DefaultGroovyMethods.asImmutable(authors)                         : authors instanceof List ? DefaultGroovyMethods.asImmutable(authors)                         : DefaultGroovyMethods.asImmutable(authors));                 }             }         }         if (args.containsKey("title")) {this.title = (String) args.get("title");} else { this.title = null;}         if (args.containsKey("publicationDate")) {             if (args.get("publicationDate") == null) {                 this.publicationDate = null;             } else {                 this.publicationDate = (Date) ((Date) args.get("publicationDate")).clone();             }         }     } }</pre> | <pre>public Book() {     this (new HashMap()); }  public int compareTo(Book other) {     if (this == other) {         return 0;     }     Integer value = 0     value = this.title &lt;=&gt; other.title     if ( value != 0) {         return value     }     value = this.publicationDate     if ( value != 0) {         return value     }     return 0 }  public static Comparator return this\$titleComparator  public static Comparator return this\$publicationDateComparator  public String toString() {     StringBuilder _result =         boolean \$toStringFirst =         _result.append("Book")         if (\$toStringFirst) {             \$toStringFirst = false         } else {             _result.append(", ")         }         _result.append(Invoke         if (\$toStringFirst) {             \$toStringFirst = false         } else {             _result.append(", ")         }         _result.append(Invoke         if (\$toStringFirst) {             \$toStringFirst = false         } else {             _result.append(", ")         }         _result.append(Invoke         result.append(")");         if (\$toString == null) {             \$toString = _result         }         return \$toString;     }      public int hashCode() {         if ( \$hashCode == 0) {             int _result = HashCo             if (! (this.getAuthors()                 _result = HashCodeHelper.updateHash(_result, this.getAuthors());             )             if (! (this.getTitle().equals(this)) {                 _result = HashCodeHelper.updateHash(_result, this.getTitle());             }             if (! (this.getPublicationDate().equals(this)) {                 _result = HashCodeHelper.updateHash(_result, this.getPublicationDate());             }             \$hashCode = (int) _result;         }         return \$hashCode;     }      public boolean canEqual(Object other) {         return other instanceof Book;     } }</pre> | <pre>public boolean equals(Object other) {     if ( other == null) {         return false;     }     if (this == other) {         return true;     }     if (! (other instanceof Book)) {         return false;     }     Book otherTyped = (Book) other;     if (! (otherTyped.canEqual(this ))) {         return false;     } }</pre> | <pre>static {     this\$titleComparator = new Book\$titleComparator();     this\$publicationDateComparator = new Book\$publicationDateComparator(); }  public String getAuthors(int index) {     return authors.get(index); }  public List&lt;String&gt; getAuthors() {     return authors; }  public final String getTitle() {     return title; }  public final Date getPublicationDate() {     if (publicationDate == null) {         return publicationDate;     } else {         return (Date) publicationDate.clone();     } }  public int compare(java.lang.Object param0, java.lang.Object param1) {     return -1; }  private static class Book\$titleComparator extends AbstractComparator&lt;Book&gt; {     public Book\$titleComparator() {} }  public int compare(Book arg0, Book arg1) {     if (arg0 == arg1) {         return 0;     }     if (arg0 != null &amp;&amp; arg1 == null) {         return -1;     }     if (arg0 == null &amp;&amp; arg1 != null) {         return 1;     }     return arg0.title &lt;=&gt; arg1.title; }  public int compare(java.lang.Object param0, java.lang.Object param1) {     return -1; }  private static class Book\$publicationDateComparator extends AbstractComparator&lt;Book&gt; {     public Book\$publicationDateComparator() {} }  public int compare(Book arg0, Book arg1) {     if ( arg0 == arg1 ) {         return 0;     }     if ( arg0 != null &amp;&amp; arg1 == null) {         return -1;     }     if ( arg0 == null &amp;&amp; arg1 != null) {         return 1;     }     return arg0 .publicationDate &lt;=&gt; arg1 .publicationDate; }  public int compare(java.lang.Object param0, java.lang.Object param1) {     return -1; } }</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Who is  
Groovy?



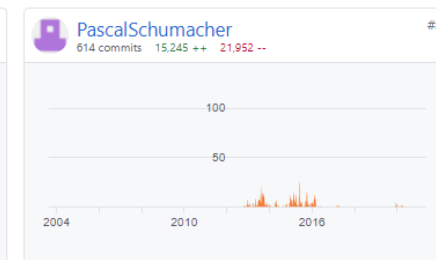
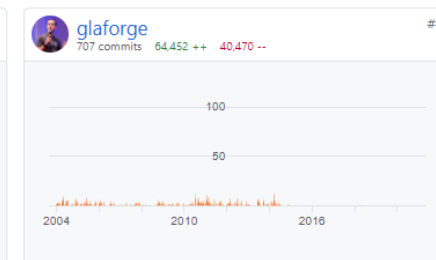
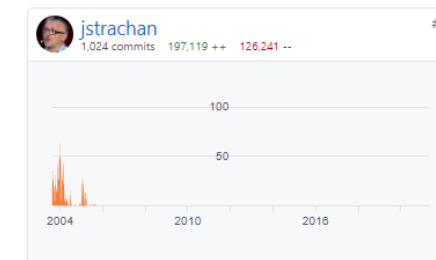
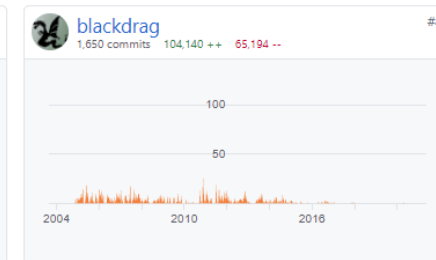
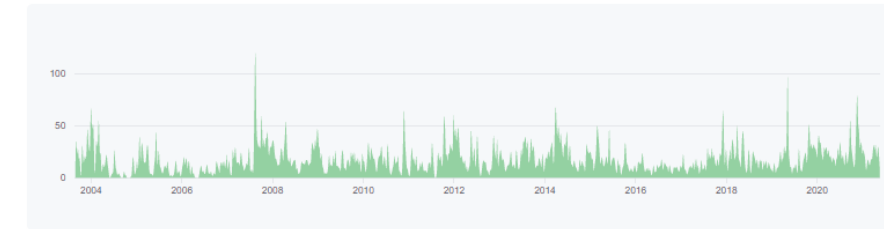
# Top Contributors



Aug 31, 2003 – May 28, 2021

Contributions: Commits

Contributions to master, excluding merge commits and bot accounts



# Top Contributors

## Groovy 1.0 released!

Posted on 03 Jan 2007

**Groovy 1.0** has finally been **released**.

It has been a long road, lasting over three years, but at last I can recommend Groovy as a stable, production proven and well documented programming language for the Java platform. (not to mention a very helpful and large **user community**)

I'll be giving a **Talk about the Groovy Compiler** on 25 Jan 2007 in London, please come along to that, and the pub afterwards, to discuss all things Groovy.

I will be at the **Groovy Developers Conference** in Paris (starts 29 Jan 2007) to discuss the future direction of Groovy, in particular enhanced **IDE support**.

In May 2007 there will be a three day **Groovy and Grails conference** at the Barbican, which will be perfect for anyone interested in this language to meet the core developers, and discover the cutting edge progress in Groovy.

But most of all, Groovy 1.0 would not be where it is today without the work of the open source community. I'd like to thank each person for their work in making Groovy a fantastic language:

### Committers:

**Guillaume Laforge** (glaforge), **Jochen Theodorou** (blackdrag), **Jeremy Rayner** (jez), **John Wilson** (tug), **Russel Winder** (russel), **James Strachan** (jstrachan), **Dierk Koenig** (dierk), **Christian Stein** (cstein), **Paul King** (paulk), **Joe Walnes** (joe), **Chris Stevenson** (skizz), **Jamie McCrindle** (jamiemc), **Matt Foemmel** (mattf), **Sam Pullara** (spullara), **Kasper Nielsen** (kasper), **Travis Kay** (travis), **Zohar Melamed** (zohar), **Bob McWhirter** (bob), **Chris Poirier** (cpoirier), **Christiaan ten Klooster** (ckl), **Steve Goetze** (goetze), **Bing Ran** (bran), **John Stump** (jstump), **Pilho Kim** (phk), **Mark Chu-Carroll** (markcc), **Alan Green** (alang), **Edward Povazan** (emp), **Franck Rasolo** (fraz), **John Rose** (jrose), **Graeme Rocher** (graeme), **Guillaume Alleon** (galleon), **Antti Karanta** (akaranta), **Dave Kerber** (davekerber), **Hein Meling** (hmeling), **Joachim Baumann** (jbaumann), **James E. Ervin** (jervin), **Scott Hickey** (jshickey), **Martin C. Martin** (mcspanky), **Marc Guillemot** (mguillem), **Aslak Hellesoy** (rinkrank), **Steven Devijver** (sdevijver), **Scott Stirling** (sstirling), **Yuri Schimke** (yuri)

### Contributors:

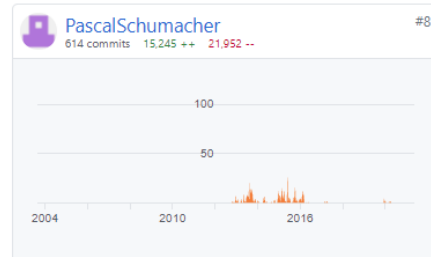
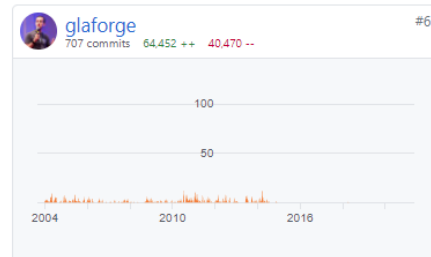
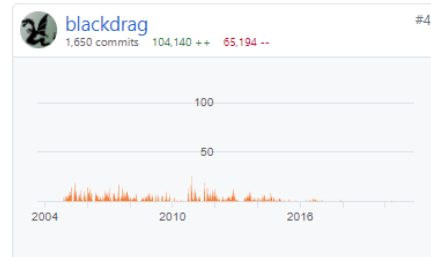
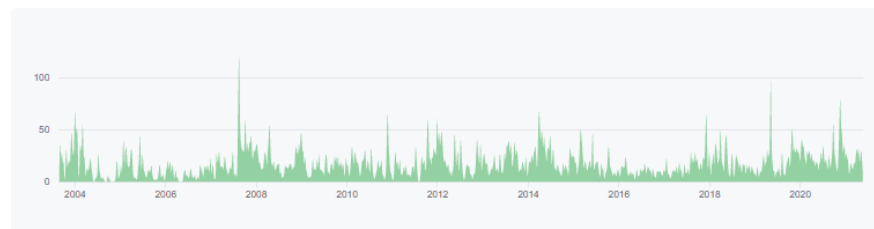
Joern Eyrich, Robert Kuzelj, Rod Cope, James Birchfield, Robert Fuller, Sergey Udovenko Hallvard Traetteberg, Peter Reilly, Brian McCallister Richard Monson-Haefel, Brian Larson, Artur Biesiadowski Ivan Z. Ganza, Arjun Nayyar, Mark Turansky, Jean-Louis Berliet, Graham Miller, Marc Palmer, Tugdual Grall, and many many many other contributors and organisations.

Thankyou All (and apologies if I missed anyone, so many people...) :-)

Aug 31, 2003 – May 28, 2021

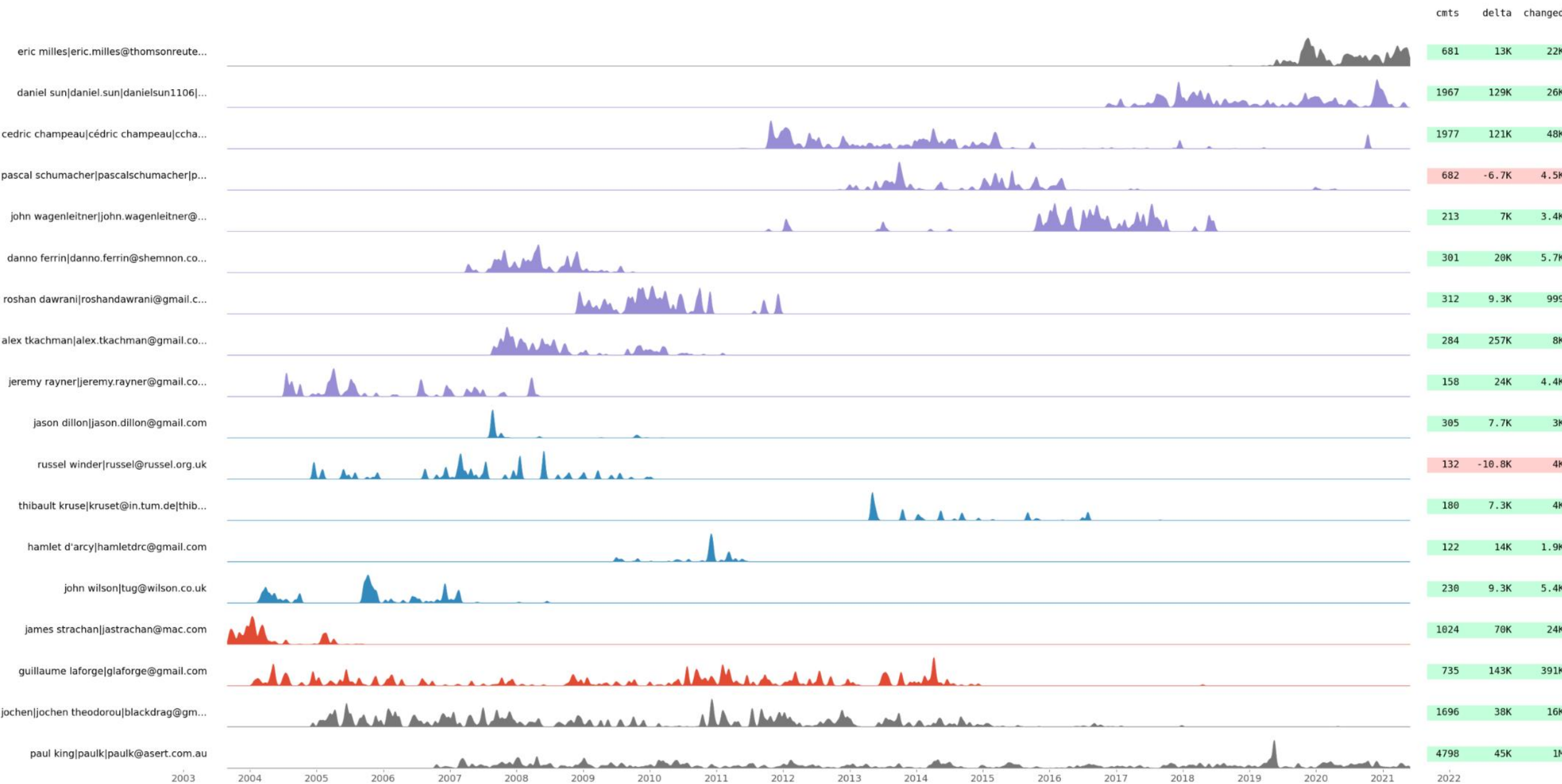
Contributions: Commits

Contributions to master, excluding merge commits and bot accounts

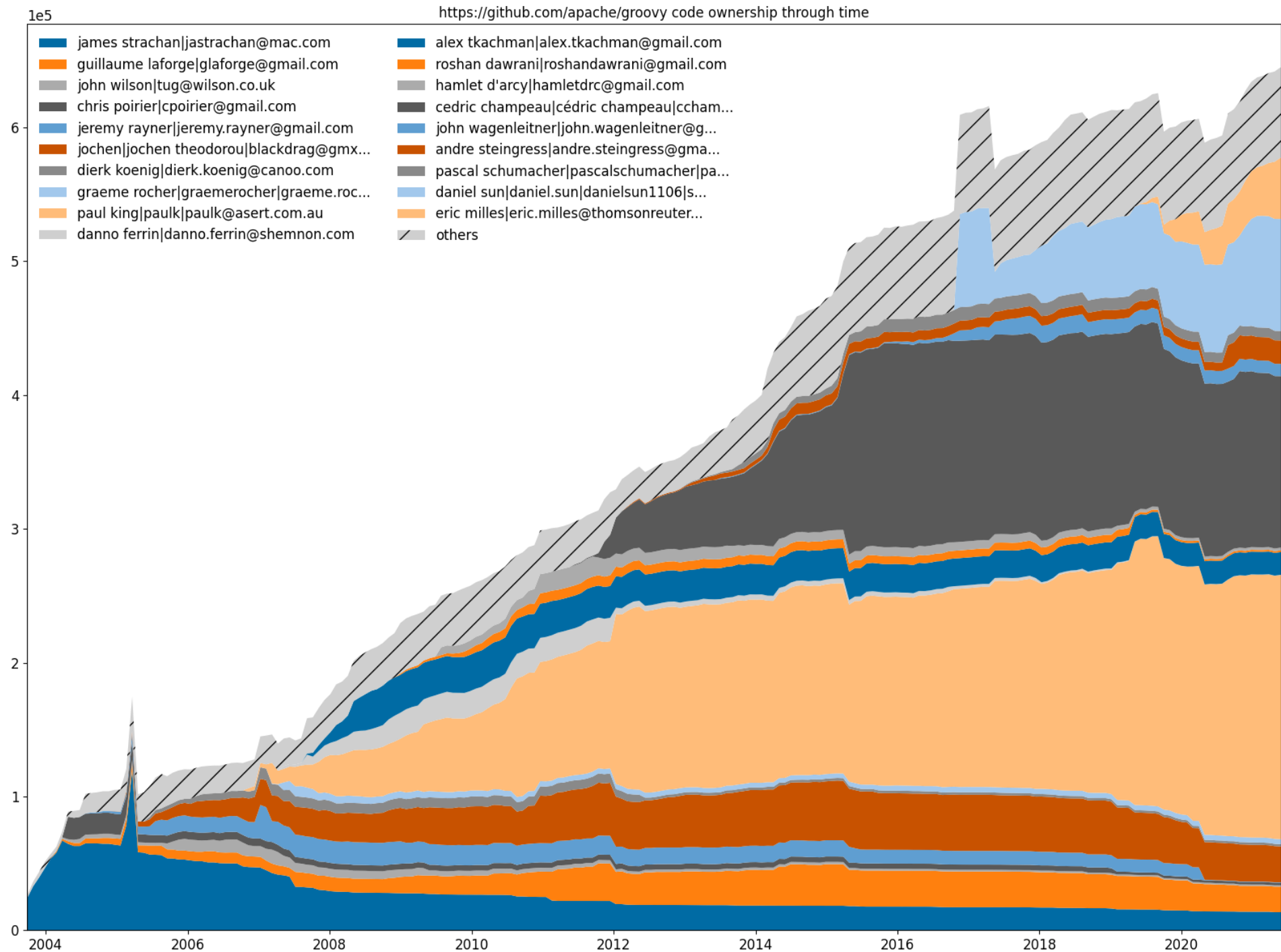




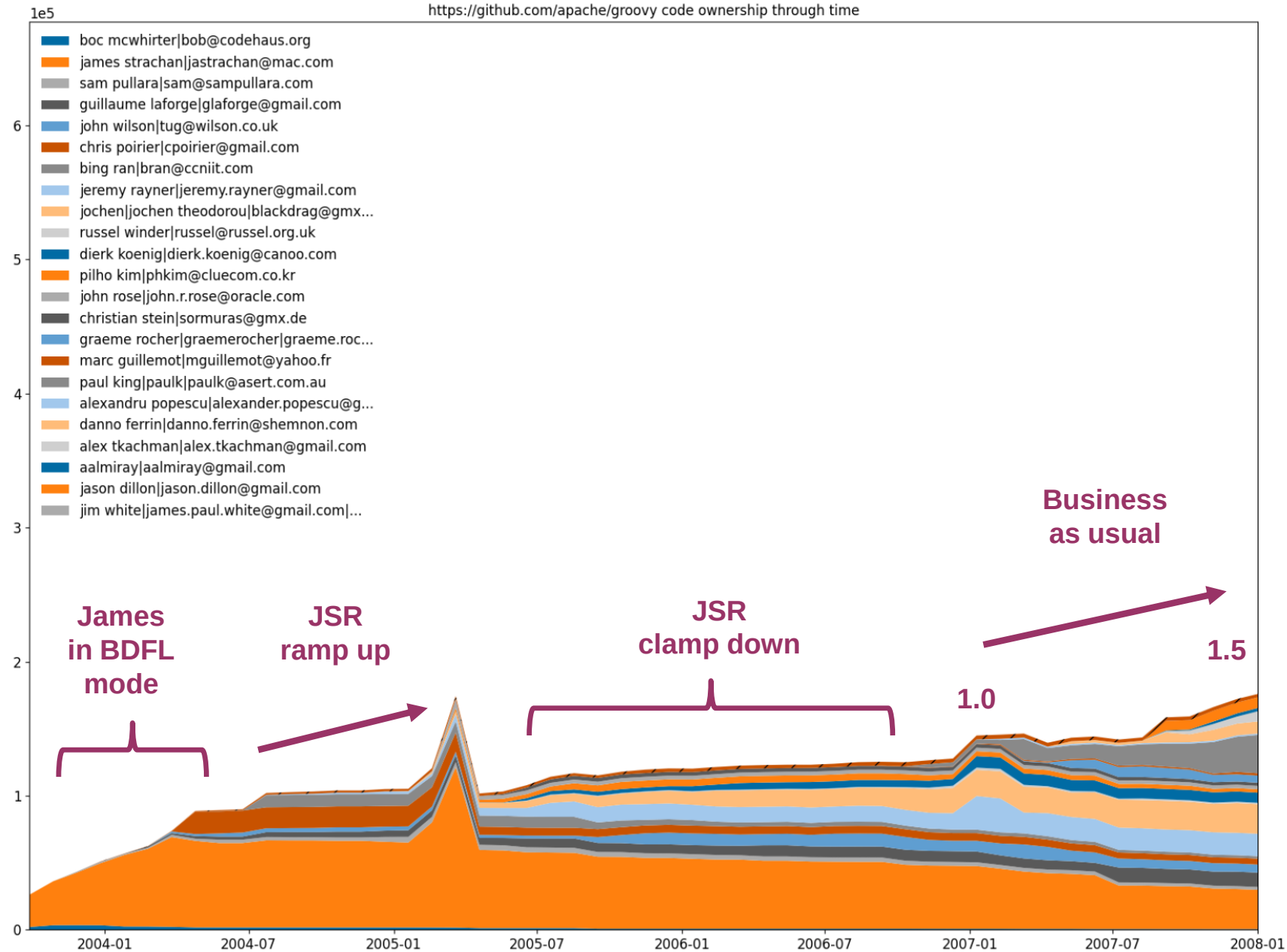
# Top Contributors



# Top Contributors



# Early history



# Distinguished contributions



## Jochen Theodorou (blackdrag)

- *Took on the role of technical lead once James departed*



## Cédric Champeau

- *Brought much needed energy to the team and was the key driver behind Groovy's static nature & traits*



## Guillaume Laforge

- *Numerous contributions to core and took over project lead when James left*

## Also

- **Jeremy Rayner:** *Initial Antlr parser*
- **John Wilson:** *Builders/parsers including lazy implementations*
- **Alex Tkachman:** *numerous contributions, optimisation & AST transforms*
- **Danno Ferrin:** *@Grab & AST transformation framework, and builders*
- **Jason Dillon:** *Groovysh repl*

# Distinguished contributions



## Paul King

- *Numerous contributions to core and took over project leadership from Guillaume*



## Daniel Sun

- *Numerous contributions to core including Parrot parser & Language integrated query*



## Eric Milles

- *Numerous contributions to core particularly for static compiler*

## Also

- **Roshan Dawrani, Pascal Schumacher, John Wagenleitner, Hamlet D'Arcy & Shil Sinha:** *Numerous contributions to core over an extended period*
- And many others ...

# Distinguished contributions – Frameworks



**Graeme Rocher:**

*Numerous contributions to Groovy core  
plus instigator of Grails and later Micronaut*

**Peter Niederwieser:**

*Instigator of Spock, contributor  
to Groovy and Gradle*



**Hans Dockter:**

*Founder of Gradle*

**Andres Almiray:**

*Instigator of Griffon, contributor  
to Groovy and Gradle*

**Also: current Spock, Geb  
and CodeNarc teams and  
many others ...**

- Frameworks were key drivers of language features





# Distinguished contributions - GParS



Václav Pech

- *Numerous contributions to Groovy core plus instigator of GParS*



Russel Winder – dearly remembered 🐿️❤️

- *Numerous contributions to Groovy core, various build technologies and GParS*



Dierk Koenig

- *Numerous contributions to Groovy core, key contributor to Gpars, and author of Groovy in Action*

# Distinguished contributions - GParS

- A Concurrency & Parallelism Framework for Groovy and Java
  - *Fork/join, parallel arrays, map/reduce, actors, agents, STM, CSP, dataflow, asynchronous functions*

```
def decryptor = actor {  
  loop {  
    react { String message ->  
      reply message.reverse()  
    }  
  }  
}  
  
def console = actor {  
  decryptor << 'lellarap si yvoorG'  
  react {  
    println 'Decrypted message: ' + it  
  }  
}  
  
console.join()
```

```
def nums = [1, 2, 3, 4]  
def result = nums.collectParallel{it * 2}  
assert result == [2, 4, 6, 8]
```



```
class Account {  
  private amount = newTxnInteger(0)  
  
  void transfer(final int a) {  
    atomic { amount.increment(a) }  
  }  
  
  int getCurrentAmount() {  
    atomicWithInt { amount.get() }  
  }  
}
```

```
def future = {it * 2}.callAsync(3)  
assert 6 == future.get()
```

*Data Parallelism:*  
*Fork/Join*  
*Map/Reduce*

*Fixed coordination*  
*(for collections)*

*Actors*

*Explicit coordination*

*Safe Agents*

*Delegated coordination*

*Dataflow*

*Implicit coordination*

```
final def df = new Dataflows()  
  
task { df.z = df.x + df.y }  
  
task { df.x = 10 }  
  
task { df.y = 5 }  
  
assert df.z == 15
```

```
def agent = new Agent([])  
agent { it << 'Dave' }  
agent { it << 'Joe' }  
assert agent.val.size() == 2
```



# The Groovy Project

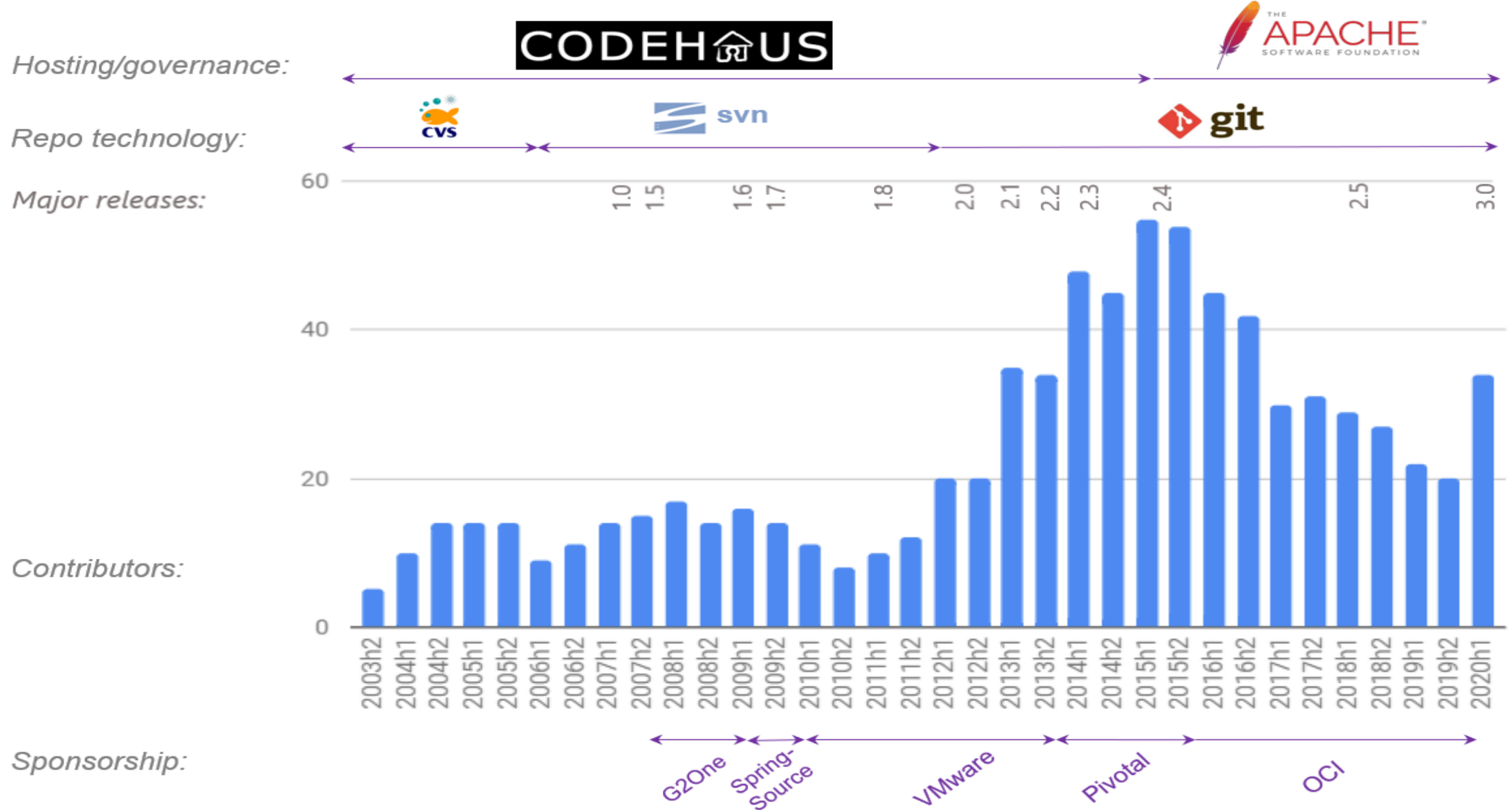
# Contributors & Direction

- Driven by those interested in fixing pain points
- Not by dislike of another language or paradigm
- Not a single person's vision (except briefly initially)
- Not an academic view of some ideal language
- Not a corporation plugging a strategic weakness
- No one company

# Governance

- Projects running at Codehaus and Apache have many similarities, but the details are different
- Both follow a collegiate style of decision making and community involvement in decisions is actively encouraged
- Apache has some more formalised structure:
  - *Formal process potentially involving voting can be used for making decisions*
  - *A Project Management Committee led by its Chair have ultimately responsibility for the project's technical direction and report to the Apache board in regard to following Apache procedures and policy*

# The project: **Contributors** and more



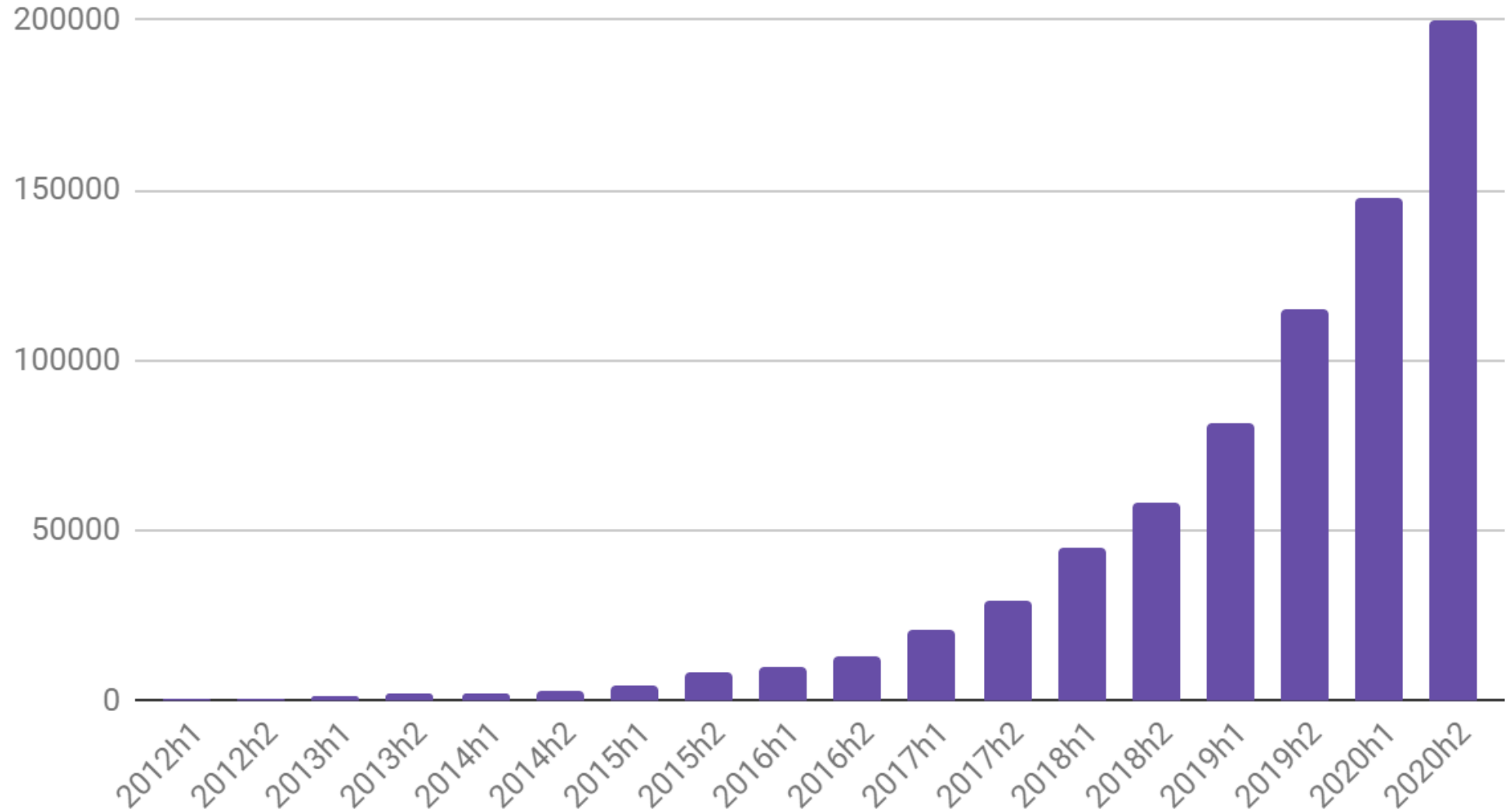
# The project: **Downloads**

Popular &  
growing

2016: 23M  
2017: 50M  
2018: 103M  
2019: 197M  
2020: 347M

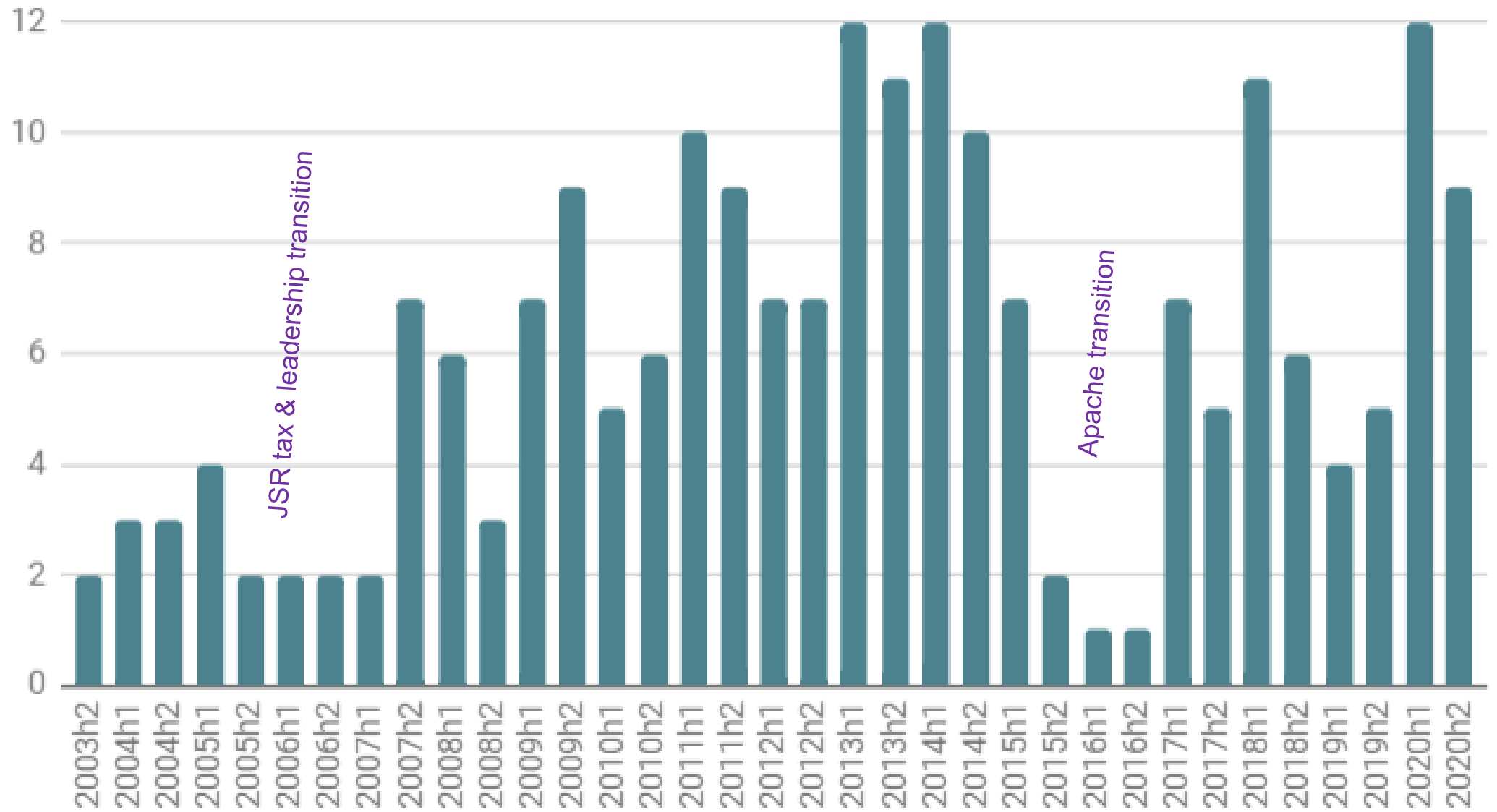
\* includes Bintray &  
Maven Central only

Downloads K's



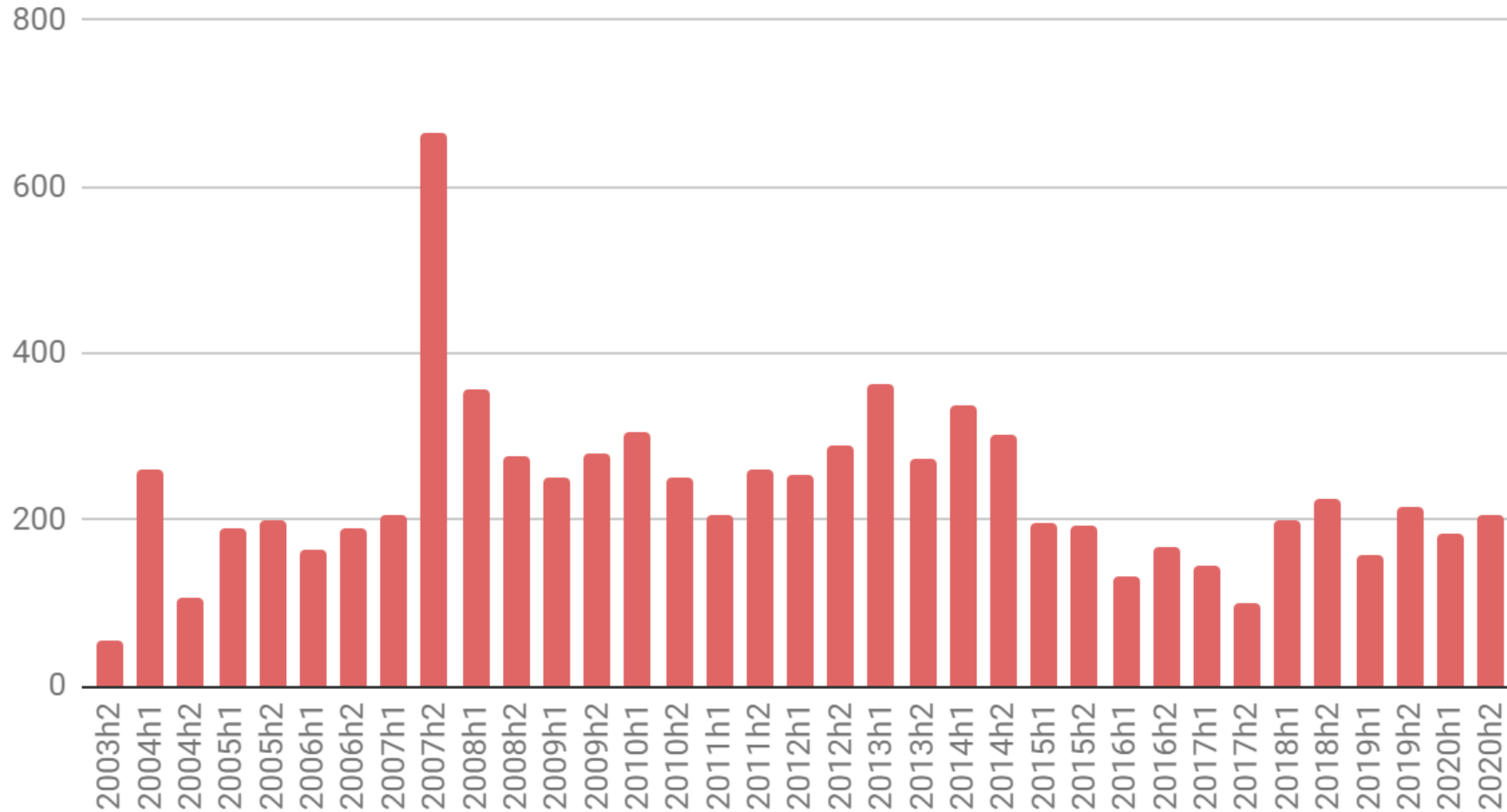
# The project: Releases

## Releases



# The project: **Issues Resolved**

Issues resolved



# The codebase

4683 text files. 4658 unique files. 179 files ignored.

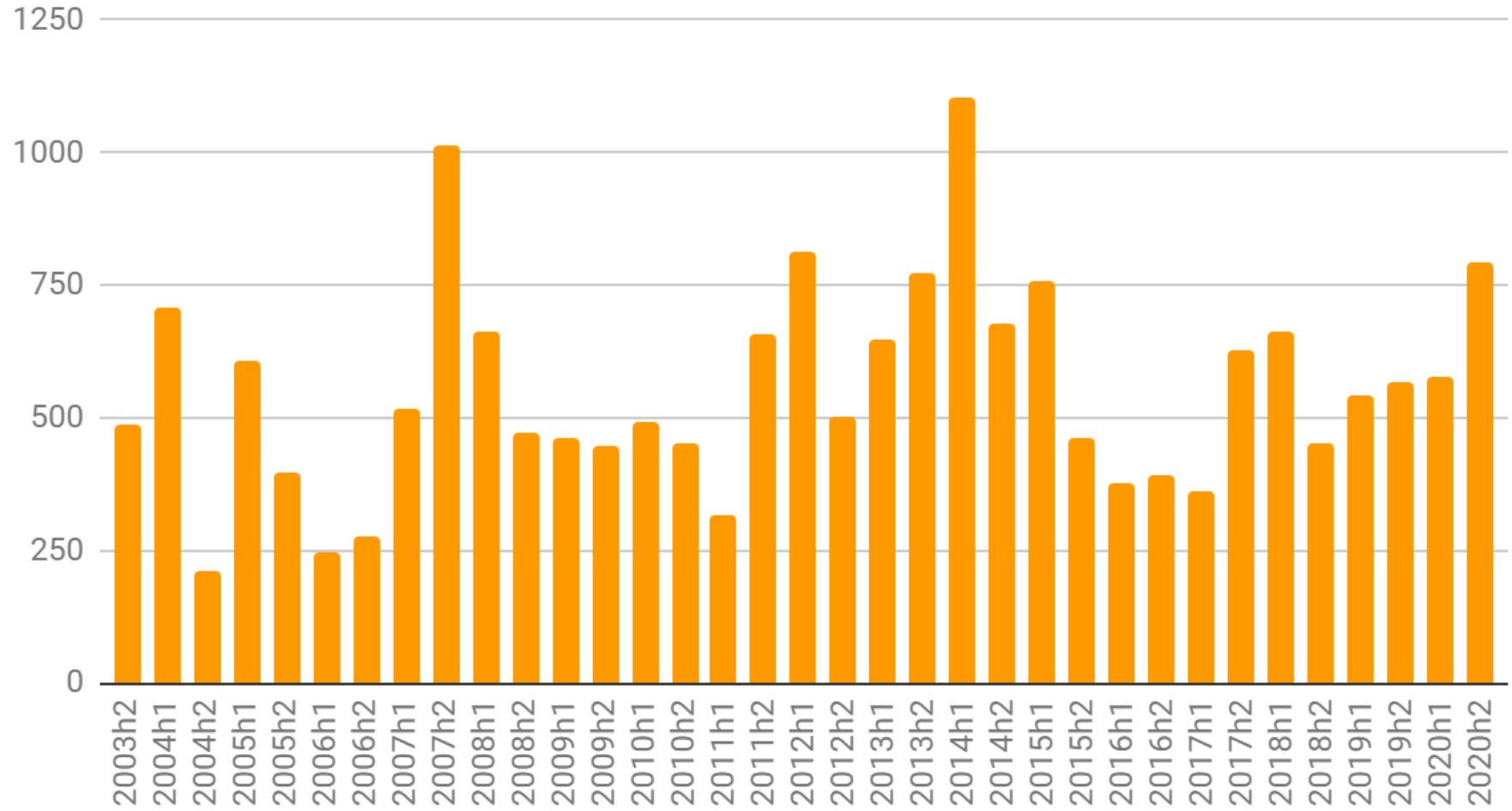
github.com/AlDanial/cloc v 1.82 T=76.65 s (58.8 files/s, 8968.7 lines/s)

| Language           | files | blank | comment | code   |
|--------------------|-------|-------|---------|--------|
| Groovy             | 2500  | 38545 | 61818   | 205931 |
| Java               | 1728  | 36239 | 97495   | 195436 |
| AsciiDoc           | 81    | 6822  | 231     | 21471  |
| CSS                | 4     | 1357  | 19      | 5176   |
| Gradle             | 64    | 375   | 1256    | 3726   |
| HTML               | 70    | 414   | 1262    | 2565   |
| XML                | 16    | 161   | 361     | 1643   |
| ANTLR Grammar      | 2     | 397   | 234     | 1603   |
| DOS Batch          | 9     | 114   | 19      | 503    |
| Bourne Shell       | 10    | 110   | 274     | 483    |
| YAML               | 9     | 41    | 128     | 244    |
| Bourne Again Shell | 6     | 70    | 451     | 223    |
| Markdown           | 3     | 23    | 0       | 68     |
| XSD                | 1     | 12    | 17      | 57     |
| JavaScript         | 2     | 3     | 16      | 18     |
| TOML               | 1     | 1     | 14      | 2      |
| SUM:               | 4506  | 84684 | 163595  | 439149 |

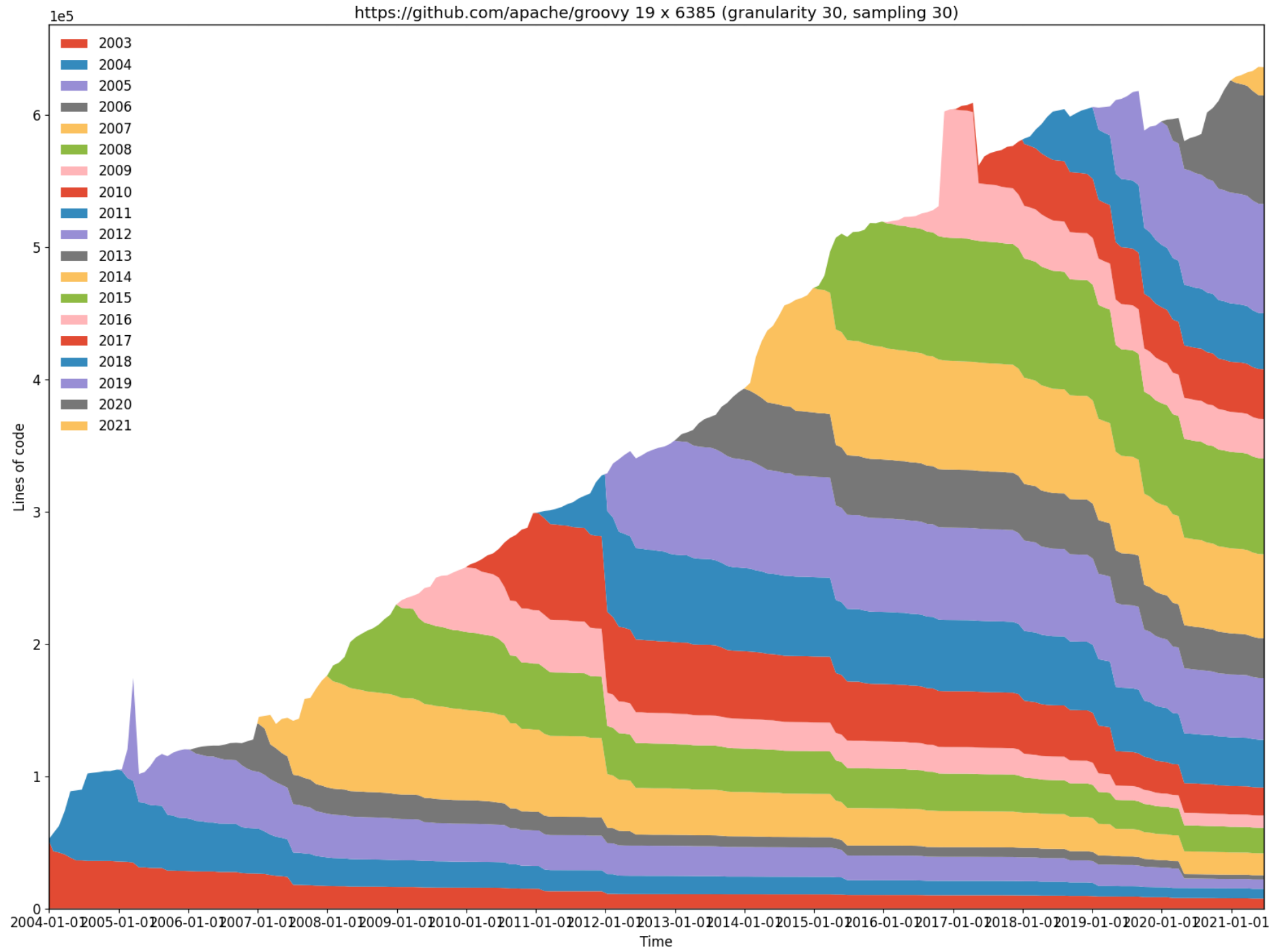


# The codebase

## Commits



# The codebase



# The project: **Highlights**

|        |                                                        |     |
|--------|--------------------------------------------------------|-----|
| 2003h2 |                                                        |     |
| 2004h1 |                                                        |     |
| 2004h2 |                                                        |     |
| 2005h1 |                                                        |     |
| 2005h2 |                                                        |     |
| 2006h1 |                                                        |     |
| 2006h2 |                                                        |     |
| 2007h1 | <i>Initial release – dynamic companion for Java</i>    | 1.0 |
| 2007h2 | <i>Elvis op, joint compiler, ExpandoMetaClass</i>      | 1.5 |
| 2008h1 |                                                        |     |
| 2008h2 |                                                        |     |
| 2009h1 | <i>Multi-assignment, grapes, <b>AST transforms</b></i> | 1.6 |
| 2009h2 | <i>Power asserts, AST browser</i>                      | 1.7 |
| 2010h1 |                                                        |     |
| 2010h2 |                                                        |     |
| 2011h1 | <b>Command chains</b> , functional enhancements        | 1.8 |
| 2011h2 |                                                        |     |
| 2012h1 |                                                        |     |
| 2012h2 | <b>Static nature</b> , modules, indy support           | 2.0 |
| 2013h1 | <i>Meta-annotations, type checker extensions</i>       | 2.1 |
| 2013h2 | <i>SAM coercion, DSL &amp; groovysh improvements</i>   | 2.2 |
| 2014h1 | <i>Closure type inference, traits, JDK8</i>            | 2.3 |
| 2014h2 |                                                        |     |
| 2015h1 | <i>Android support, many small improvements</i>        | 2.4 |
| 2015h2 |                                                        |     |
| 2016h1 |                                                        |     |
| 2016h2 |                                                        |     |
| 2017h1 |                                                        |     |
| 2017h2 |                                                        |     |
| 2018h1 | <i>AST reworking, macros, macro methods</i>            | 2.5 |
| 2018h2 |                                                        |     |
| 2019h1 |                                                        |     |
| 2019h2 |                                                        |     |
| 2020h1 | <b>Parrot parser</b> (inc lambdas), split packaging    | 3.0 |

# Interesting Aspects

# Interesting aspects

- Flow sensitive typing
- Extensible type system
- Script integration

# Flow-sensitive typing

# Flow-sensitive typing

## Dynamic Groovy

```
def o = 'foobarbaz'  
o = o.toUpperCase()    // String  
o = o.size()           // int  
o = Math.sqrt(o)       // double  
assert o == 3d
```

↑  
Runtime  
type

# Flow-sensitive typing

## Dynamic Groovy

```
def o = 'foobarbaz'  
o = o.toUpperCase()    // String  
o = o.size()           // int  
o = Math.sqrt(o)       // double  
assert o == 3d
```

Runtime  
type



```
def o = 'foobarbaz'  
o = o.size()  
o = o.toUpperCase()
```

groovy.lang.MissingMethodException:  
No signature of method: java.lang.Integer.toUpperCase()  
is applicable for argument types: () values: []



# Flow-sensitive typing

## Static Groovy

```
def o = 'foobarbaz'  
o = o.toUpperCase()    // String  
o = o.size()           // int  
o = Math.sqrt(o)       // double  
assert o == 3d
```

Inferred  
type through  
Flow Typing



- Groovy's type checker enhanced to be able to check such code (2011)
- Later found prior art in Whiley (2010) and related research in Typed Scheme (2008).

# Flow-sensitive typing

## Static Groovy

```
def o = 'foobarbaz'  
o = o.toUpperCase() // String  
o = o.size() // int  
o = Math.sqrt(o) // double  
assert o == 3d
```

- Groovy's type checker enhanced to be able to check such code (2011)
- Later found prior art in Whiley (2010) and related research in Typed Scheme (2008).

```
def o = 'foobarbaz'  
o = o.toUpperCase()  
o = o.size()  
o = Math.sqrt(o)  
assert o == 3d
```

ExpressionStatement - Binary

- Binary - (o = o.toUpperCase())

ExpressionStatement - Binary

- Binary - (o = o.size())

ExpressionStatement - Binary

- Binary - (o = java.lang.Math.sqrt(o))

BlockStatement - (2)

|                 |                          |
|-----------------|--------------------------|
| lastLineNumber  | 5                        |
| leftExpression  | org.codehaus.groovy.ast. |
| lineNumber      | 5                        |
| metaDataMap     | [INFERRED_TYPE:int]      |
| nodeMetaData    | [INFERRED_TYPE:int]      |
| operation       | ("=" at 5.3: "-" )       |
| rightExpression | org.codehaus.groovy.ast. |

# Flow-sensitive typing

## Static Groovy

```
def o = 'foobarbaz'  
o = o.toUpperCase()    // String  
o = o.size()           // int  
o = Math.sqrt(o)       // double  
assert o == 3d
```

Inferred  
type through  
Flow Typing



```
def o = 'foobarbaz'  
o = o.size()  
o = o.toUpperCase()
```

[Static type checking] - Cannot find matching  
method int#toUpperCase()

# Flow-sensitive typing

## A more interesting example

```
def component = someCondition ? new ArrayDeque() : new Stack()
component.clear() // LUB: AbstractCollection or Serializable or Cloneable
if (component instanceof ArrayDeque) {
    component.addFirst(1) // only in ArrayDeque
} else if (component instanceof Stack) {
    component.addElement(2) // only in Stack
}
if (component instanceof ArrayDeque || component instanceof Stack) {
    println component.peek() // checked duck typing
}
```

# Flow-sensitive typing

## A more interesting example

```
def component = someCondition ? new  
component.clear() // LUB: AbstractCollection of Serializable or Comparable  
if (component instanceof ArrayDeque) {  
    component.addFirst(1) // only in ArrayDeque  
} else if (component instanceof Stack) {  
    component.addElement(2) // only in Stack  
}  
if (component instanceof ArrayDeque || component instanceof Stack) {  
    println component.peek() // checked duck typing  
}
```

```
// JEP 394: Pattern matching for instanceof  
if (component instanceof ArrayDeque ad) {  
    ad.addFirst(1);  
} else if (component instanceof Stack s) {  
    s.addElement(2);  
}
```

'||' is not conducive to Java 16 improvements:  
JEP 394: Pattern Matching for instanceof

# Extensible type checking

# Recall: “Better scripting” example

```
1 import org.junit.Test;
2 import java.util.regex.Matcher;
3 import java.util.regex.Pattern;
4 import static org.junit.Assert.*;
5
6 public class RegexTest {
7     @Test
8     public void testRegex() {
9         Pattern pattern = Pattern.compile("a:1 b:2 c:3");
10        Matcher matcher = pattern.matcher("a:1 b:2 c:3");
11        int match = 0;
12        while (matcher.find()) {
13            match++;
14            if (match == 2) {
15                String full = matcher.group(0);
16                assertEquals("b:2", full);
17                String key = matcher.group(1);
18                assertEquals("b", key);
19                String value = matcher.group(2);
20                assertEquals("2", value);
21            }
22        }
23        assertEquals(3, match);
24    }
25 }
```

Listing 1. Java regular expression matching

```
1 def (first, second, third) = 'a:1 b:2 c:3' =~ /(\S+):(\S+)/
2 def (full, key, value) = second
3 assert full == 'b:2' && key == 'b' && value == '2'
```

Listing 2. Groovy regular expression matching

- Regex find operator syntactic sugar
- Regex friendly string variants
- Multi-assignment
- First class assertions
- Indexing syntactic sugar
- Operator overloading
- Library enhancements
- Extensions/metaprogramming

# Better scripting: error handling

```
import org.junit.Test;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import static junit.framework.Assert.assertEquals;

public class RegexTest {
    @Test
    public void testRegexMatching() {
        Pattern regex = Pattern.compile("(\\S+):(\\S+)");
        Matcher matcher = regex.matcher("a:1 b:2 c:3");
        int match = 0;
        while (matcher.find()) {
            match++;
            if (match == 2) {
                String full = matcher.group(0);
                assertEquals("b:2", full);
                String key = matcher.group(1);
                assertEquals("b", key);
                String value = matcher.group(2);
                assertEquals("2", value);
            }
        }
        assertEquals(3, match);
    }
}
```



# Better scripting: error handling

```
import org.junit.Test;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import static junit.framework.Assert.assertEquals;

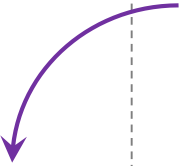
public class RegexTest {
    @Test
    public void testRegexMatching() {
        Pattern regex = Pattern.compile("(\\S+):(\\S+)");
        Matcher matcher = regex.matcher("a:1 b:2 c:3");
        int match = 0;
        while (matcher.find()) {
            match++;
            if (match == 2) {
                String full = matcher.group(0);
                assertEquals("b:2", full);
                String key = matcher.group(1);
                assertEquals("b", key);
                String value = matcher.group(2);
                assertEquals("2", value);
            }
        }
        assertEquals(3, match);
    }
}
```

# Better scripting: error handling

```
import org.junit.Test;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import static junit.framework.Assert.assertEquals;

public class RegexTest {
    @Test
    public void testRegexMatching() {
        Pattern regex = Pattern.compile("(\\S+):(\\S+)");
        Matcher matcher = regex.matcher("a:1 b:2 c:3");
        int match = 0;
        while (matcher.find()) {
            match++;
            if (match == 2) {
                String full = matcher.group(0);
                assertEquals("b:2", full);
                String key = matcher.group(1);
                assertEquals("b", key);
                String value = matcher.group(2);
                assertEquals("2", value);
            }
        }
        assertEquals(3, match);
    }
}
```

Error scenario:  
Missing closing round bracket



# Better scripting: error handling

```
import org.junit.Test;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import static junit.framework.Assert.assertEquals;

public class RegexTest {
    @Test
    public void testRegexMatching() {
        Pattern regex = Pattern.compile("(\\S+):(\\S+");
        Matcher matcher = regex.matcher("a:1 b:2 c:3");
        int match = 0;
        while (matcher.find()) {
            match++;
            if (match == 1) {
                assertEquals("b:2", full);
                String key = matcher.group(1);
                assertEquals("b", key);
                String value = matcher.group(2);
                assertEquals("2", value);
            }
        }
        assertEquals(3, match);
    }
}
```

Error scenario:  
Missing closing round bracket

Leads to runtime exception:

java.util.regex.PatternSyntaxException: Unclosed group near index 10  
(\\S+):(\\S+

# Optional type checker extension: regex checker

```
import org.junit.Test;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import static junit.framework.Assert.assertEquals;
```

```
public class RegexTest {
```

```
    @Test
```

```
    public void testRegexMatching() {
```

```
        Pattern regex = Pattern.compile("(\\S+):(\\S+)");
```

```
        Matcher matcher = regex.matcher("a:1 b:2 c:3");
```

```
        int match = 0;
```

```
        while (matcher.find()) {
```

```
            match++;
```

```
            if (match == 2) {
```

```
                String full = matcher.group(0);
```

```
                assertEquals("b:2", full);
```

```
                String key = matcher.group(1);
```

```
                assertEquals("b", key);
```

```
                String value = matcher.group(2);
```

```
                assertEquals("2", value);
```

```
            }
```

```
        }
```

```
        assertEquals(3, match);
```

```
    }
```

```
}
```

```
def matcher = 'a:1 b:2 c:3' =~ /(\\S+):(\\S+)
```

[Static type checking] - Bad regex: Unclosed group near index 10  
(\\S+):(\\S+

# Optional type checker extension: regex checker

```
~/\w{3/           // missing closing repetition quantifier brace
~"(. )o(. *"  
Pattern.compile(/?/) // dangling meta character '?' (Java Longhand)

'foobar'  =~ /f[o]{2/           // missing closing repetition quantifier brace
'foobar' ==~ /(foo/             // missing closing group bracket
Pattern.matches(/?/, 'foo')     // dangling meta character '?' (Java Longhand)

def m = 'foobar' =~ /(...)(...)/
assert m[0][1] == 'foo'         // okay
assert m[0][3]                 // type error: only two groups in regex

Pattern p = Pattern.compile('(...)(...)')
Matcher m = p.matcher('foobar')
assert m.find()
assert m.group(1) == 'foo'      // okay
assert m.group(3)              // type error: only two groups in regex
```

# Script Integration

# Script integration

```
class Robot {  
    void move(Direction dir) {  
        println "robot moved $dir"  
    }  
}
```

```
enum Direction {  
    left, right, forward, backward  
}
```

```
import static Direction.*  
  
def robot = new Robot()  
robot.move(left)
```

# Script integration

```
abstract class RobotBaseScript extends Script {  
    void move(Direction dir) {  
        this.binding.robot.move dir  
    }  
}  
  
def binding = new Binding(robot: new Robot())  
  
def importCustomizer = new ImportCustomizer()  
importCustomizer.addStaticStars Direction.name  
def transformCustomizer = new ASTTransformationCustomizer(CompileStatic)  
  
def config = new CompilerConfiguration()  
config.addCompilationCustomizers importCustomizer, transformCustomizer  
config.scriptBaseClass = RobotBaseScript.name
```

```
import static Direction.*
```

```
def robot = new Robot()  
robot.move(left)
```

```
def shell = new GroovyShell(binding, config)  
shell.evaluate '''  
    move left  
'''
```



# Script integration

```
abstract class RobotBaseScript extends Script {  
    void move(Direction dir) {  
        this.binding.robot.move dir  
    }  
}  
  
def binding = new Binding(robot: new Robot())  
  
def importCustomizer = new ImportCustomizer()  
importCustomizer.addStaticStars Direction.name  
def transformCustomizer = new ASTTransformationCustomizer(CompileStatic)  
  
def config = new CompilerConfiguration()  
config.addCompilationCustomizers importCustomizer, transformCustomizer  
config.scriptBaseClass = RobotBaseScript.name
```

```
import static Direction.*
```

```
def robot = new Robot()  
robot.move(left)
```

```
def shell = new GroovyShell(binding, config)  
shell.evaluate '''  
    move left  
'''
```

# Restrospective

# Java Compatibility

## Pros

- Minimise learning curve
- Evolve a language with relatively small contributor base
  - *Piggy back on Java improvements*
- Evolve with reduced spec requirements
  - *Fallback could be Java behaviour in many situations*

## Cons

- Constant catch up
- Sometimes Java enhancement conflicts with how we fixed the “pain point” earlier
- Limits radical changes

# Java compatibility

|                                    | 2006 | 2007 | 2008 | 2009 | 2010 | 2011 | 2012 | 2013 | 2014 | 2015 | 2016 | 2017 | 2018 | 2019 | 2020 | 2021 | Justification/Workaround |    |    |    |    |                                          |
|------------------------------------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|--------------------------|----|----|----|----|------------------------------------------|
| <b>Java</b>                        | 6    |      |      |      |      | 7    |      |      | 8    |      |      |      | 9    | 10   | 11   | 12   | 13                       | 14 | 15 | 16 | 17 |                                          |
| <b>Groovy</b>                      |      | 1    | 1.5  |      | 1.6  | 1.7  |      | 1.8  | 2    | 2.1  | 2.2  | 2.3  | 2.4  |      | 2.5  |      | 3                        |    |    |    | 4? |                                          |
| <i>varargs</i>                     |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    |                                          |
| <i>static imports</i>              |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    |                                          |
| <i>enums</i>                       |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Write enum in Java                       |
| <i>generics</i>                    |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    |                                          |
| <i>classic for loop</i>            |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Groovy has better alternatives           |
| <i>annotation definitions</i>      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Write in Java                            |
| <i>inner classes</i>               |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Closures often better                    |
| <i>array initializers</i>          |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use Groovy's literal list notation       |
| <i>underscores in numbers</i>      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use pre-existing syntax                  |
| <i>multi-catch</i>                 |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Expand manually                          |
| <i>binary literals</i>             |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Write using other base                   |
| <i>ARM in try catch</i>            |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Groovy has better alternatives           |
| <i>lambdas</i>                     |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use Closures + SAM coercion              |
| <i>method references</i>           |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use method Closures                      |
| <i>type annotations</i>            |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use multiple targets                     |
| <i>repeating annotations</i>       |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Source annotations could be repeated     |
| <i>var reserved type</i>           |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use "def" instead                        |
| <i>var lambda params</i>           |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use "def" instead                        |
| <i>switch expressions</i>          |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use Groovy switch                        |
| <i>text blocks</i>                 |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use Groovy equivalents                   |
| <i>records</i>                     |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Write records in Java or use @RecordType |
| <i>instanceof pattern matching</i> |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    | Use dynamic Groovy/flow typing           |
| <i>sealed Classes</i>              |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |                          |    |    |    |    |                                          |

Not supported

Supported by an alternative feature

Preview

Partially supported

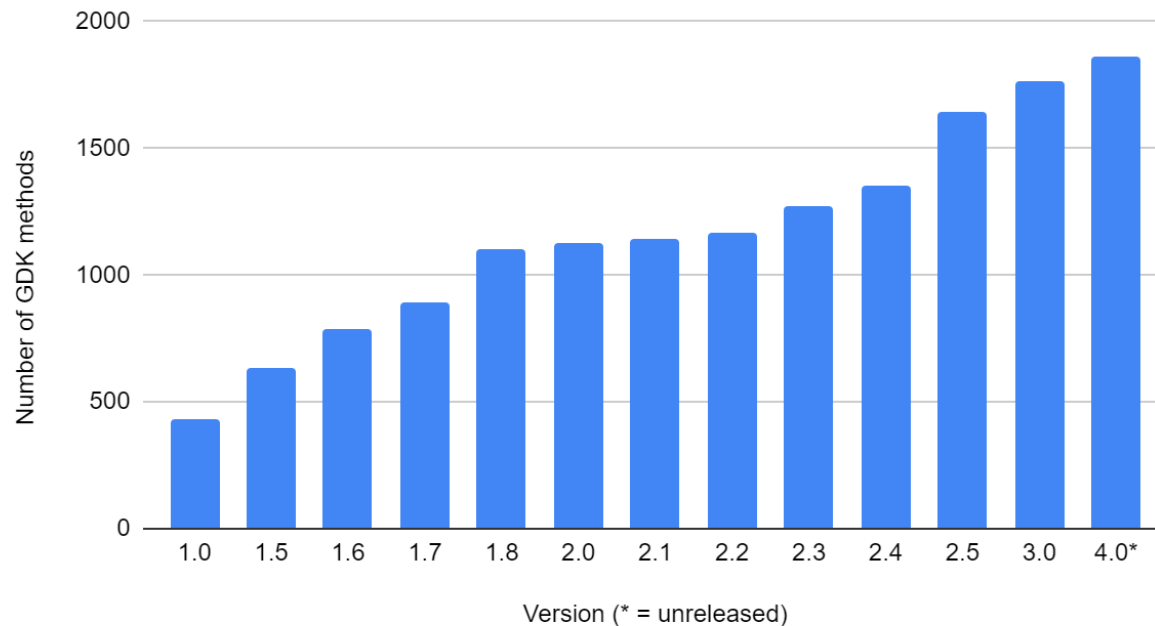
Full support

# Retrospective: Can we keep enhancing Java?

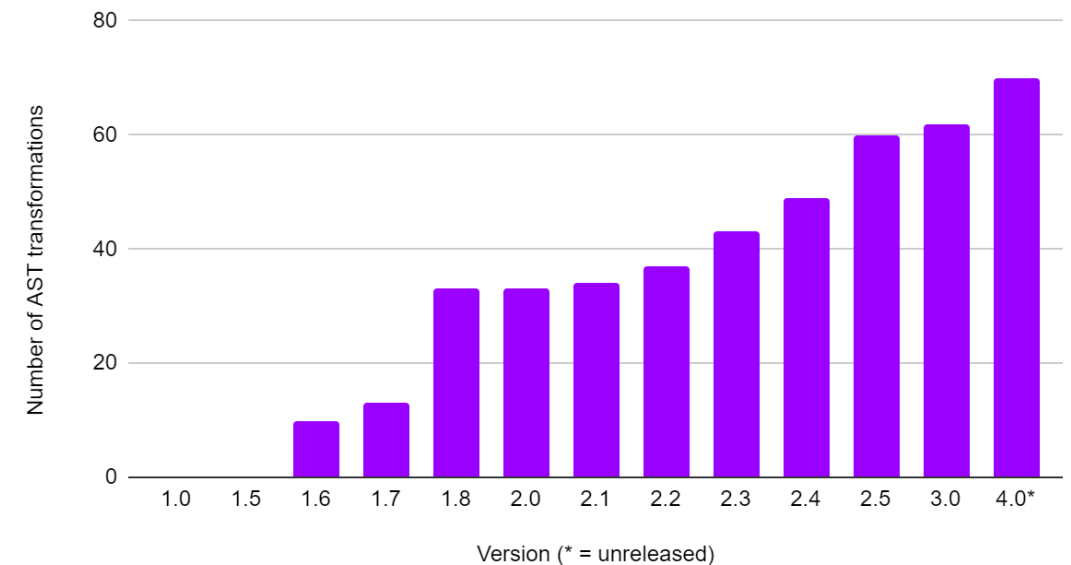
```
File.metaClass.getWordCount = {  
    delegate.text.split(/\w+/).size()  
}
```

```
@Immutable(copyWith = true)  
@Sortable(excludes = 'authors')  
@AutoExternalize  
class Book {  
    @IndexedProperty List<String> authors  
    String title  
    Date publicationDate  
}
```

GDK methods vs. Version



AST transforms vs. Version



# Influencing other languages

Languages introducing features similar to Groovy at a later date include:

- The dangling closure syntax is used by Scala, Kotlin, and Swift
- The safe navigation operator is used by Swift, Ruby, C#, Kotlin, CoffeeScript, and others
- Range notation is used by Swift
- The Spaceship operator, somewhat inspired from Perl, is now also used by Php, Ceylon, and Ruby
- The Elvis operator `'? :'` is also used by Fantom, Kotlin, Gosu, Xtend, and Ruby. Swift and C# use `'??'`
- Flow-sensitive typing is also used by Ceylon, Kotlin, Facebook Flow, and TypeScript

# Influencing other languages

## Kotlin borrows from Groovy in numerous:

- Groovy's dangling closures inspired Kotlin's dangling lambdas
- Groovy's builder concept inspired Kotlin's type-safe builders
- Kotlin's default 'it' parameter for lambdas was borrowed from Groovy
- Kotlin's string template syntax followed Groovy syntax used in its interpolated strings and templates
- Kotlin's Elvis operator was inspired by Groovy's similar operator
- Kotlin's null-safe call was inspired by Groovy's null-safe navigation operator (but with a different approach for non/nullable types)